

Introduction To Web Development And Django

Web Application: The applications which will provide services over the web are called web applications.

Eg: gmail.com, facebook.com, etc.

Every web application contains 2 main components:

- 1) Front-End
- 2) Back-End

1) Front-End: It represents what user is seeing on the website.

We can develop Front-End content by using the following technologies: HTML, JS, CSS, JQuery and Bootstrap.

JQuery and Bootstrap are advanced front-end technologies, which are developed by using HTML, CSS and JavaScript only.

HTML: HyperText Markup Language Every web application should contain HTML. i.e HTML is the mandatory technology for web development. It represents structure of web page.

CSS: Cascading Style Sheets It is optional technology, still every web application contains CSS. The main objective of CSS is to add styles to the HTML Pages like colors, fonts, borders etc.

Java Script: It allows to add interactivity to the web application including programming logic. The main objective of Java Script is to add functionality to the HTML Pages.

i.e to add dynamic nature to the HTML Pages. HTML Meant for Static Responses HTML+JS Meant for Dynamic Responses

Static Response vs Dynamic Response:

- If the response is not varied from time to time and person to person then it is considered as static response.
Eg: Login Page of Gmail, Home Page of ICICI Bank
- If the response is varied from time to time and person to person then it is considered as dynamic response.
Eg: Inbox Page of Gmail, Balance Page of ICICI Bank

2) Back End: It is the technology used to decide what to show to the end user on the Front-End.

i.e Backend is responsible to generate required response to the end user, which is displayed by the Front-End.

Back-End has 3 important components:

- i) The Language like Java, Python etc
- ii) The Framework like Django, Pyramid, Flask etc
- iii) The Database like SQLite, Oracle, MySQL etc

For the Backend language Python is the best choice b'z of the following reasons:

- (i) Simple and easy to learn, libraries and concise code.
- (ii) For the Framework Django is the best choice b'z it is Fast, Secure and Scalable.
- (iii) Django is the most popular web application framework for Python.
- (iv) Django provides inbuilt database which is nothing but SQLite, which is the best choice for database.
- (v) The following are various popular web applications which are developed by using Python and Django
YouTube, Instagram, Dropbox, Disqus, Spotify, Quora, pinterest, Reddit etc

Django:

- 🏠 Django is a free and open-source web framework.
- 🏠 It is written in Python.
- 🏠 It follows the Model-View-Template (MVT) architectural pattern.
- 🏠 It is maintained by the Django Software Foundation (DSF)
- 🏠 It is used by several top websites like
Youtube, Google, Dropbox, Yahoo Maps, Mozilla, Instagram, Washington Times, Nasa and many more
- 🏠 Django was created in 2003 as an internal project at Lawrence Journal-World News Paper for their web development.
- 🏠 The Original authors of Django Framework are: Adrian Holovaty, Simon Willison

⚽ After Testing this framework with heavy traffics, Developers released for the public as open source framework on July 21st 2005.

⚽ The Django was named in the memory of Guitarist Django Reinhardt.

⚽ Official website: djangoproject.com

Django Project vs Django Application:

-> A Django project is a collection of applications and configurations which forms a full web application.

Eg: Bank Project

-> A Django Application is responsible to perform a particular task in our entire web application.

Eg: loan app registration app polling app etc

Project = Several Applications + Configuration Information

Note:

1) The Django applications can be plugged into other projects. ie these are reusable. (Pluggable Django Applications)

2) Without existing Django project there is no chance of existing Django Application. Before creating any application first we required to create project.

How to create Django Project:

Once we installed django in our system, we will get 'django-admin' command line tool, which can be used to create our Django project.

django-admin startproject ProjectName

D:\>mkdir.djangoprojects

D:\>cd.djangoprojects

D:\djangoprojects>django-admin start-project firstProject

The following project structure will be created

D:\ldjangoprojects>

```
|
+---firstProject
|
|---manage.py
|
+---firstProject
|
|---settings.py
|
|---urls.py
|
|--wsgi.py
|
|--_init_.py
```

init_.py:

It is a blank python script. Because of this special file name, Django treated this folder as python package.

Note: If any folder contains `_init_.py` file then only that folder is treated as Python package. But this rule is applicable until Python 3.3 Version.

settings.py:

In this file we have to specify all our project settings and configurations like installed applications, middleware configurations, database configurations etc

urls.py:

- Here we have to store all our url-patterns of our project.
- For every view (web page), we have to define separate url-pattern. End user can use url-patterns to access our webpages.

wsgi.py:

- wsgi à Web Server Gateway Interface.
- We can use this file while deploying our application in production on online server.

manage.py:

- The most commonly used python script is manage.py
- It is a command line utility to interact with Django project in various ways like to run development server, run tests, create migrations etc.

How to Run Django Development Server:

We have to move to the manage.py file location and we have to execute the following command.

```
py manage.py runserver
```

```
D:\djangoprojects\firstProject>py manage.py runserver
```

Performing system checks...

System check identified no issues (0 silenced).

server at <http://127.0.0.1:8000/> Quit the server with CTRL-BREAK.

Now the server started.

How to Send First Request:

Open browser and send request:

<http://127.0.0.1:8000/>

The following should be response if everything goes fine.

It worked!

Congratulations on your first Django-powered page.

Next, start your first app by running `python manage.py startapp [app_label]`.

You're seeing this message because you have `DEBUG = True` in your Django settings file and you haven't configured any URLs. Get to work!

Role of Web Server:

- Web Server provides environment to run our web applications.
- Web Server is responsible to receive the request and forward request to the corresponding web component based on url-pattern and to provide response to the end user.
- Django framework is responsible to provide development server. Even Django framework provides one inbuilt database sqlite. Special Thanks to Django.

Note: Once we started Server a special database related file will be generated in our project folder structure.

db.sqlite3

Creation of First Web Application:

Once we creates Django project, we can create any number of applications in that project.

The following is the command to create application.

```
python manage.py startapp firstApp
```

```
D:\djangoprojects\firstProject>python manage.py
```

```
startapp firstApp
```

The following is the folder structure got created.

```
D:\djangoprojects>
```

```
└── firstProject
```

```
    | db.sqlite3
```

| manage.py

|

|——firstApp

| | admin.py

| | apps.py

| | models.py

| | tests.py

| | views.py

| | _init_.py

| |

| L——migrations

| _init_.py

|

L——firstProject

| settings.py

| urls.py

| wsgi.py

| _init_.py

Note: Observe that Application contains 6 files and project contains 4 files+ one special file: manage.py

1) __init__.py:

It is a blank Python script. Because of this special name, Python treated this folder as a package.

2) admin.py:

We can register our models in this file. Django will use these models with Django's admin interface.

3) apps.py:

In this file we have to specify application's specific configurations.

4) models.py:

In this file we have to store application's data models.

5) tests.py:

In this file we have to specify test functions to test our code.

1) views.py:

In this file we have to save functions that handles requests and return required responses.

2) Migrations Folder:

This directory stores database specific information related to models.

Note: The most important commonly used files in every project are
views.py and models.py

Activities required for Application:

Activity-1: Add our application in settings.py, so that Django aware
about our application.

In **settings.py**:

```
1) INSTALLED_APPS = [  
    'Application_name',  
]
```

Activity-2: Create a view for our application in views.py.

- View is responsible to prepare required response to the end user. i.e view contains business logic.
- There are 2 types of views.
 - 1) Function Based Views
 - 2) Class Based Views
- In this application we are using Function based views.

views.py:

```
1) from django.shortcuts import render  
2) from django.http import HttpResponse  
3) # Create your views here.  
  
def display(request):  
    s='<h1>Hello Students welcome to Django classes!!!</h1>'  
    return HttpResponse(s)
```

- 1) Each view will be specified as one function in views.py.
- 2) In the above example display is the name of function which is nothing but one view.
- 3) Each view should take atleast one argument (request)
- 4) Each view should return HttpResponseRedirect object with our required response.

View can accept request as input and perform required operations and provide proper response to the end user.

Activity-3: Define url-pattern for our view in urls.py file.

- This url-pattern will be used by end-user to send request for our views.
- The 'urlpatterns' list routes URLs to views.

For functional views we have to do the following 2 activities:

- 1) Add an import: from firstApp import views
- 2) Add a URL to urlpatterns: url('greeting/', views.display)

URL.py

```
from django.contrib import admin
from django.urls import path
from Visual import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('welcome/', views.UI),
]
```

Whenever end user sending the request with urlpattern: greeting then dispaly() function will be executed and provide required response.

Activity-4: Start Server and Send the request py manage.py runserver <http://127.0.0.1:8000/greetings>

Http Request flow in Django Application:

1. Whenever end user sending the request first Django development server will get that request.
2. From the Request django will identify urlpattern and by using urls.py, the corresponding view will be identified.
3. The request will be forwarded to the view. The corresponding function will be executed and provide required response to the end user.

Summary of Sequence of Activities related to Django Project:

1) Creation of Django project

django-admin startproject firstProject

2) Creation of Application in that project py manage.py startapp firstApp

3)Add application to the Project (inside settings.py)

4) Define view function inside views.py

5) Define url-pattern for our view inside urls.py

6) Start Server py manage.py runserver

7) Send the request

How to change Django Server Port:

- By default Django development server will run on port number: 8000. But we can change port number based on our requirement as follows.

`py manage.py runserver 7777`

Now Server running on port number: 7777

We have to send the request with this port number only

<http://127.0.0.1:7777/greetings/>
<http://127.0.0.1:8000/time/>

Various Practice Applications:

1. Write Django Application just to send Helloworld message as response.
2. Write Django application to send server time as response
3. Single application with multiple views

views.py:

1) `from django.shortcuts import render`

2) `from django.http import HttpResponse`

3)

4) `# Create your views here.`

5) `def good_morning_view(request):`

6) `return HttpResponse('<h1>Hello Friend Good Morning!!!</h1>')`

7)

```
8) def good_evening_view(request):
```

```
9)     return HttpResponse('<h1>Hello Friend Good Evening !!!</h1>')
```

```
10)
```

```
11) def good_afternoon_view(request):
```

```
12)     return HttpResponse('<h1>Hello Friend Good Afternoon!!!</h1>')
```

urls.py

```
1) from django.conf.urls import url
```

```
2) from django.contrib import admin
```

```
3) from testapp import views
```

```
4)
```

```
5) urlpatterns = [
```

```
6)     url('admin/', admin.site.urls),
```

```
7)     url('morning/', views.good_morning_view),
```

```
8)     url('afternoon/', views.good_afternoon_view),
```

```
    ]
```

Defining URL Patterns at Application Level instead of Project Level:

A Django project can contain multiple applications and each application can contain multiple views. Defining url-patterns for all views of all applications inside urls.py file of project creates maintenance problems and reduces reusability of applications.

We can solve this problem by defining url-patterns at application level instead of project level. For every application we have to create a separate urls.py file and we have to define all that application specific urls in that file. We have to link this application level urls.py file to project level urls.py file by using include() method.

Demo Application:

1. Creation of Project

```
django-admin startproject urlProject
```

2. Creation of Application

```
py manage.py startapp testapp
```

3. Add our application to the Project inside settings.py file

```
INSTALLED_APPS=[
```

```
.....
```

```
    'testapp'
```

```
]
```

4. Define View Function in views.py

views.py

```
1) from django.shortcuts import render
```

```
2) from django.http import HttpResponse
```

3)

4) **# Create your views here.**

5) **def first_view(request):**

6) **return HttpResponse('<h1>Response from First View</h1>')**

7)

8) **def second_view(request):**

9) **return HttpResponse('<h1>Response from Second View</h1>')**

10)

11) **def third_view(request):**

12) **return HttpResponse('<h1>Response from third View</h1>')**

13)

14) **def fourth_view(request):**

15) **return HttpResponse('<h1>Response from Fourth View</h1>')**

16)

17) **def fifth_view(request):**

18) **return HttpResponse('<h1>Response from Fifth View</h1>')**

5. Create a separate urls.py file inside application

testapp/urls.py

1) from django.conf.urls import url

2) from testapp import views

3)

4) urlpatterns = [

5) url('first', views.first_view),

6) url('second', views.second_view),

7) url('third', views.third_view),

8) url('fourth', views.fourth_view),

9) url('fifth', views.fifth_view),

10)]

6. Include this application level urls.py inside project level urls.py file.

from django.conf.urls import include

project level:urls.py

1) from django.conf.urls import url,include

2) from django.contrib import admin

4) urlpatterns = [

5) url('admin', admin.site.urls),


```
6) url('testapp/', include('testapp.urls')),  
7) ]
```

7. Run Server

```
py manage.py runserver
```

8. Send Request

[http://127.0.0.1:8000/
urlApp/test](http://127.0.0.1:8000/urlApp/test)

Note: We can see reusability of application in other project just with only 2 lines addition

1. settings.py -->add application name
2. urls.py --->just add: url('urlApp/',include('urlApp.urls')),

Advantages:

The main advantages of defining urlpatterns at application level instead of project level are

- 1) It promotes reusability of Django Applications across multiple projects
- 2) Project level urls.py file will be clean and more readable

Django Templates:

It is not recommended to write html code inside python script (views.py file) because:

- 1) It reduces readability because Python code mixed with html code
- 2) No separation of roles. Python developer has to concentrate on both python code and HTML Code.

3) It does not promote reusability of code

We can overcome these problems by separating html code into a separate html file. This html file is nothing but a template.

From the Python file (views.py file) we can use these templates based on our requirement.

We have to write templates at project level only once and we can use these in multiple applications.

Python Stuff required to develop Template Based Application:

1) To know the current Python file name `print(__file__)` #test.py

2) To know absolute path of current

Python File Name `import os`

`print(os.path.abspath(__file__))`

Output: D:\Djangoclasses\test.py

3) To know Base Directory name of the current file

`print(os.path.dirname(os.path.abspath(__file__)))`

Output: D:\Djangoclasses

4) Inside D:\Djangoclasses there is one folder named with templates. To know its absolute path

Note: The main advantage of this approach is we are not required to hard code system specific paths (locations) in our python script.

Steps to develop Template Based Application:

1) Creation of Project

`django-admin startproject templateProject`

2) Creation of Application

`py manage.py startapp testApp`

3) Add this application to the project in settings.py file,so that Django aware of application

4) Create a 'templates' folder inside main project folder.

In that templates folder create a seperate folder named with testApp to hold that particular application specific templates.

5) Add templates folder to settings.py file so that Django can aware of our templates.

```
TEMPLATES = [  
    {  
        ...  
        'DIRS': ['D:\djangoprojects\templateProject\templates'],  
    },]
```

It is not recommended to hard code system specific locations in settings.py file. To overcome this problem, we can generate templates directory path programatically as follows.

```
import os  
  
BASE_DIR =  
os.path.dirname(os.path.dirname(os.path.abspath( file )))  
TEMPLATE_DIR =BASE_DIR/'templates'
```

Specify this TEMPLATE_DIR inside settings.py as follows

```
TEMPLATES = [  
    {  
        ...  
        'DIRS': [TEMPLATE_DIR],  
    },]
```

6) Create html file inside templateProject/templates/testApp folder. This html file is nothing but template.

wish.html

1) <!DOCTYPE html>

2) <html lang="en" dir="ltr">

3) <head>

4) <meta charset="utf-8">

5) <title>First Template Page</title>

6) </head>

7) <body>

8) <h1>Hello welcome to Second Hero of MVT: Templates</h1>

9) </body>

10) </html>

7) Define Function based view inside views.py file

```
from django.shortcuts import render
```

```
    return render(request, 'testApp/wish.html')
```

8) Define URL Pattern either at application level or at project level.

9) Run Server and send Request.

Template Tags:

From Python views.py we can inject dynamic content to the template file by using template tags.

Template Tags also known as Template Variables.

Take special care about Template tag syntax it is not python syntax and not html syntax. Just it is special syntax.

Template Tag Syntax for inserting Text Data: {{insert_date}}

This template tag we have to place inside template file (ie html file) and we have to provide insert_date value from python views.py file.

Templates Based Applications:

1. Application to Display Hello World
2. Application To display date and time
3. Application To display date ,time and student information
4. Application to display wish message based on time like Good Morning,Good Evening etc

Note: Try to get Basic Idea about MVT Design Pattern

Working with Static Files:

☕ Up to this just we injected normal text data into template by using template tags.

☕ But sometimes our requirement is to insert static files like images,css files etc inside template file.

Process to include Static Files inside Template:

- 1) Create a folder named with 'static' inside main project folder. It is exactly same as creating 'templates' folder.
- 2) In that 'static' folder create 'images' folder to place image files.
- 3) Add static directory path to settings.py file, so that Django can aware of our images.

settings.py

1) `STATIC_DIR=BASE_DIR/'static'`

2)

3)

4) `STATIC_URL = '/static/'`

5)

6) `STATICFILES_DIRS=[`

7) `STATIC_DIR,`

8) `]`

- 4) Make sure all paths are correct or not

<http://127.0.0.1:8000/static/images/divine3.jpg>

5) Use Template Tags to insert image

At the beginning of HTML just after `<!DOCTYPE html>` we have to include the following template tag `{% load static %}`

Just we are conveying to the Django to load all static files.

We have to include image file as follows ``

wish.html

1) `<!DOCTYPE html>`

2) `{% load staticfiles %}`

3) `<html lang="en" dir="ltr">`

4) `<head>`

5) `<meta charset="utf-8">`

6) `<title>First Template Page</title>`

7) `<style >`

8) `#h1{`

9) `color:red;`

10) `}`

11) `#h2{`

12) `color:green;`

13) }

14) </style>

15) </head>

16) <body>

17) <h1 id=h11>{{insert_msg}}</h1>

18) <h1 id=h12>Current Date and Time : {{insert_date}}</h1>

19) <h1>This climate preferable image is:</h1>

20)

21) </body>

22) </html>

views.py

1) from django.shortcuts import render

2) import datetime

3)

4) # Create your views here.

5) def wish(request):

6) date=datetime.datetime.now()


```
7)      msg=None

8)      h=int(date.strftime('%H'))

9)      if h<12:

10)     msg='Hello Guest !!!! Very Very Good Morning!!!!'

11)     elif h<16:

12)     msg='Hello Guest !!!! Very Very Good AfterNoon!!!!'

13)     elif h<21:

14)     msg='Hello Guest !!!! Very Very Good Evening!!!!'

15)     else:

16)     msg='Hello Guest !!!! Very Very Good Night!!!!'

17)     my_dict={'insert_date':date,'insert_msg':msg}

18)     return render(request,'testApp/wish.html',my_dict)
```

How to include CSS File:

1) Create a folder 'css' inside static and place our demo.css file in that css folder.

demo.css

1) img{

2) height: 500px;

3) **width:** 500px;

4) **border:** 10px red groove;

5) **margin:**0% 20%;

6) }

7) h1{

8) **color:**blue;

9) **text-align:** center;

}

2) In the template html file we have to include this css file. We have to do this by using link tag inside head tag.

```
<link rel="stylesheet" href="{% static "css/demo.css" %}" >
```

Working with Models and Databases:



As the part of web application development, compulsory we required to interact with database to store our data and to retrieve our stored data.



Django provides a big in-built support for database operations. Django provides one inbuilt database sqlite3.



For small to medium applications this database is more enough. Django can provide support for other databases also like oracle, mysql, postgresql etc

Database Configuration:



Django by default provides sqlite3 database. If we want to use this database, we are not required to do any configurations.



The default sqlite3 configurations in settings.py file are declared as follows.

settings.py

```
1) DATABASES = {  
2)     'default': {  
  
3)     'ENGINE': 'django.db.backends.sqlite3',  
  
4)     'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),  
  
5)     }
```

How to Check Django Database Connection:

☕ We can check whether django database configurations are properly configured or not by using the following commands from the shell

☕ D:\djangoprojects\modelProject>py manage.py shell

```
>>> from django.db import connection
```

```
>>> c = connection.cursor()
```

☕ If we are not getting any error means our database configurations are proper.

Model Class:

☕ A Model is a Python class which contains database information.

☕ A Model is a single, definitive source of information about our data. It contains fields and behavior of the data what we are storing.

☕ Each model maps to one database table.

☕ Every model is a Python class which is the child class of (django.db.models.Model)

☕ Each attribute of the model represents a database field.

☕ We have to write all model classes inside 'models.py' file.

1. Create a project and application and link them.

```
django-admin startproject
modelProject cd
modelProject/
python manage.py startapp testApp
```

After creating a project and application, in the models.py file, write the following code:

models.py

- 1) `from django.db import model`
- 3) `# Create your models here.`
- 4) `class Employee(models.Model):`
- 6) `ename=models.CharField(max_length=30)`
- 8) `eaddr=models.CharField(max_length=30)`

Note: This model class will be converted into Database table. Django is responsible for this.

table_name: appName_Employee

fields: eno, ename, esal and eaddr. And one extra field: id

behaviors: eno is of type Integer, ename is of type Char and max_length is 30 characters.

Hence, Model Class = Database Table Name + Field Names + Field Behaviors

Converting Model Class into Database specific SQL Code:

Once we write Model class, we have to generate the corresponding SQL Code. For this, we have to use “makemigrations” command.

Python manage.py make migrations

It results the following Migrations for ‘Testapp’:

testApp/migrations/0001_initial.py

How to see corresponding SQL Code of Migrations:

To see the generated SQL Code, we have to use the following command “sqlmigrate”

python manage.py sqlmigrate testApp 0001

1) **BEGIN;**

2) --

3) -- **Create model Employee**

4) --

5) **CREATE TABLE "testApp_employee" ("id" integer NOT NULL
PRIMARY KEY AUTOIN CREMENT, "eno" integer NOT NULL,
"ename" varchar(30) NOT NULL, "esal" real N**

OT NULL, "eaddr" varchar(30) NOT NULL);

6) **COMMIT;**

Note: Here 0001 is the file passed as an argument

“id” field:

1) For every table(model), Django will generate a special column named with “id”.

2) ID is a Primary Key. (Unique Identifier for every row inside table is considered as a primary key).

3) This field(id) is auto increment field and hence while inserting data, we are not required to provide data for this field.

4) This id field is of type “AutoField”

5) We can override the behavior of “id” field and we can make our own field as “id”.

6) Every Field is by default “NOT NULL”.

How to execute generated SQL Code (migrate Command):

After generating sql code, we have to execute that sql code to create table in database.

For this, we have to use 'migrate' command.

python manage.py migrate

Operations to perform:

Apply all migrations: admin, auth, contenttypes, sessions, testApp

Running migrations

- Applying contenttypes.0001_initial... OK
- Applying auth.0001_initial... OK
- Applying admin.0001_initial... OK
- Applying admin.0002_logentry_remove_auto_add... OK
- Applying contenttypes.0002_remove_content_type_name... OK
- Applying auth.0002_alter_permission_name_max_length... OK
- Applying auth.0003_alter_user_email_max_length... OK
- Applying auth.0004_alter_user_username_opts... OK
- Applying auth.0005_alter_user_last_login_null... OK

Note: Now tables will be created in the database.

What is the Advantage of creating Tables with 'migrate' Command

If we use 'migrate' command, then all Django required tables will be created in addition to our application specific tables. If we create table manually with sql code, then only our application specific table will be created and django my not work properly. Hence it is highly recommended to create tables with 'migrate' command.

How to Check created Table in Django admin Interface:

We have to register model class in 'admin.py' file.

admin.py

1) `from django.contrib import admin`

2) `from testApp.models import Employee`

3)

4) `# Register your models here.`

5)

6) `admin.site.register(Employee)`

Creation of Super User to login to admin Interface:

☕ We can create super user with the following command by providing username, mailid, password.

python manage.py createsuperuser

☕ We can login to admin interface → Start the server and login to admin interface using the created credentials.

☕ **python manage.py runserver**

☕ Open the following in browser: <http://127.0.0.1:8000/admin/>

Difference between makemigrations and migrate:

“makemigrations” is responsible to generate SQL code for Python model class whereas “migrate” is responsible to execute that SQL code so that tables will be created in the database.

To Display Data in admin Interface in Browser:

models.py

1) **from** django.db **import** models

2)

3) **# Create your models here.**

4)

5) **class** Employee(models.Model):

6) eno=models.IntegerField()

7) ename=models.CharField(max_length=30)

8) esal=models.FloatField()

```
9)     eaddr=models.CharField(max_length=30)

10) def _str_(self):
```

admin.py

```
1) from django.contrib import admin
3)

5)

7) list_display=['eno','ename','esal','eaddr']

9) admin.site.register(Employee,EmployeeAdmin)
```

Note:

We should do this registration in a single line otherwise we are getting error.
`admin.site.register(Employee)`

`admin.site.register(EmployeeAdmin)`

Views.py

```
1) from django.shortcuts import render

2) from testApp.models import Employee

3)
```

4) **# Create your views here.**

5) **def empdata(request):**

6) **emp_list=Employee.objects.all()**

7) **my_dict={'emp_list':emp_list}**

8) **return render(request, 'testApp/emp.html', context=my_dict)**

emp.html

1) **<!DOCTYPE html>**

2) **{% load static %}**

3) **<html lang="en" dir="ltr">**

4) **<head>**

5) **<meta charset="utf-8">**

6) **<title></title>**

7) **<link rel="stylesheet" href="{% static '/css/demo.css'%}">**

8) **</head>**

9)

10) **<body>**

11) `<h1> The employees list is : </h1>`

12)

13) `{% if emp_list %}`

14) `<table>`

15) `<thead>`

16) `<th> eno </th>`

17) `<th> ename </th>`

18) `<th> esal </th>`

19) `<th> eaddr </th>`

20) `</thead>`

21)

22) `{% for emp in emp_list %}`

23) `<tr>`

24) `<td> {{emp.eno}} </td>`

25) `<td>{{emp.ename}}</td>`

26) `<td>{{emp.esal}}</td>`

27) `<td> {{emp.eaddr}}</td>`

28) `</tr>`

29) `{% endfor %}`

30)

31) `</table>`

32) `{%else%}`

33) `<p> No records found </p>`

36) `</bod>`

37) `</html>`

Summary of Sequence of Activities related to Django

Project for Models:

1) Activate the virtual Environment: `venv\Scripts\activate`

2) Creation of Django project:

django-admin startproject firstProject

Next we need to Enter into the project level:

cd projectName

3) Creation of Application in that project:

py manage.py startapp Appname

4) Add application to the Project (inside [settings.py](#))

5) Create the Template and Static Files in the Project Level

Template Folder -> testapp -> html file

Static Folder -> css-> style.css

6) Add the configuration in [Setting.py](#) for Template and static folder

7) Inside [Models.py](#) define the table by using class and Inherit Model class

i.e class TableName(models.Model):

8) Makemigrations and migrate command:

py [manage.py](#) makemigrations

py [manage.py](#) migrate

9) Define view function inside [views.py](#) and send data to template in the form of dictionary.

10) Define url-pattern for our view inside urls.py

11) Start Server py manage.py runserver

12) Send the request by using url pattern

Working with Django Forms

Django Forms:

☕ It is the very important concept in web development.

☕ The main purpose of forms is to take user input.

Eg: login form, registration form, enquiry form etc

☕ From the forms we can read end user provided input data and we can use that data based on requirement. We may store in the database for future purpose. We may use just for validation/authentication purpose etc

☕ Here we have to use Django specific forms but not HTML forms.

Advantages of Django Forms over HTML Forms:

- 1) We can develop forms very easily with python code
- 2) We can generate HTML Form widgets/components (like textarea, email, pwd etc) very quickly
- 3) Validating data will become very easy
- 4) Processing data into python data structures like list, set etc will become easy
- 5) Creation of Models based on forms will become easy etc

Process to generate Django Forms:

Step-1: Creation of forms.py file in our application folder with our required fields.

forms.py:

- 1) `from django import forms`
- 2) `class StudentForm(forms.Form):`
- 3) `name=forms.CharField()`
- 4) `marks=forms.IntegerField()`

Note: name and marks are the field names which will be available in html form

Step-2: usage of forms.py inside views.py file:

views.py file is responsible to send this form to the template html file

views.py:

- 1) `from django.shortcuts import render`
- 2) `from . import forms`
- 3)
- 4) **# Create your views here.**
- 5) `def studentinputview(request):`
- 6) `form=forms.StudentForm()`

Alternative Short Way:

- 1) `def studentinputview(request):`

3) `return render(request, 'testapp/input.html', {'form': form})`

Note: context parameter is optional. We can pass context parameter value directly without using keyword name 'context'

Step-3: Creation of html file to hold form:

Inside template file we have to use template tag to inject form `{{form}}`

It will add only form fields. But there is no `<form>` tag and no submit button. Even the fields are not arranged properly. It is ugly form.

We can make proper form as follows

1) `<h1>Registration Form</h1>`

2) `<div class="container" align="center">`

3) `<form method="post">`

4) `{{form.as_p}}`

5) `<input type="submit" class="btn btn-primary" name="" value="Submit">`

6) `</form>`

7) `</div>`

input.html:

1) `<!DOCTYPE html>`

2) `{%load staticfiles%}`

3) `<html lang="en" dir="ltr">`

4) `<head>`

5) `<meta charset="utf-8">`

7) `<link rel="stylesheet" href="{%static 'css/demo2.css'%}">`

8) `<title></title>`

9) `</head>`

10) `<body>`

11) `<h1>Registration Form</h1>`

12) `<div class="container" align="center">`

13) `<form method="post">`

14) `{{form.as_p}}`

15) `<input type="submit" class="btn btn-primary" name="" value="Submit">`

16) `</form>`

17) `</div>`

19) `</html>`

If we submit this form we will get 403 status code
response Forbidden (403)

CSRF verification failed. Request aborted.

Help

Reason given for failure:

CSRF token missing or incorrect.

CSRF (Cross Site Request Forgery) Token:

- Every form should satisfy CSRF (Cross Site Request Forgery) Verification, otherwise Django won't accept our form.
- It is meant for website security. Being a programmer we are not required to worry anything about this. Django will take care of everything.
- But we have to add `csrf_token` in our form.

1) `<h1>Registration Form</h1>`

2) `<div class="container" align="center">`

3) `<form method="post">`

4) `{{form.as_p}}`

5) `{% csrf_token %}`

6) `<input type="submit" class="btn btn-primary" name="" value="Submit">`

7) `</form>`

8) `</div>`

If we add `csrf_token` then in the generated form the following hidden field will be added, which makes our post request secure

If we configured `csrf_token` in HTML form then only Django will accept our form.

How to process Input Data from the form inside views.py

File: We required to modify `views.py` file. The end user provided input is available in a dictionary named with '`cleaned_data`' in `views.py`:

views.py:

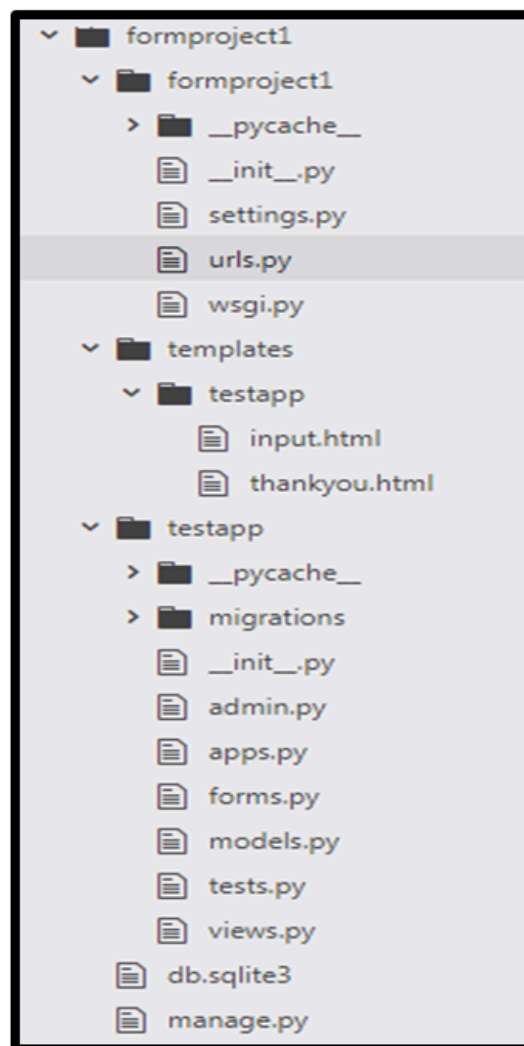
```
from django.shortcuts import render
from . import forms
```

```

# Create your views here.
def studentinputview(request):
    form=forms.StudentForm()
    if request.method=='POST':
        form=forms.StudentForm(request.POST)
        if form.is_valid():
            print('Form validation success and printing data')
            print('Name:',form.cleaned_data['name'])
            print('Marks:',form.cleaned_data['marks'])
        return render(request,'testapp/input.html',{'form':form})

```

Student Name and Marks Submission Form Project (formproject1):



forms.py

1) `from django import forms`

```
4)
marks=forms.IntegerField()
```

3) `name=forms.CharField()`

views.py

1) `from django.shortcuts import render`

```
2) from testapp.forms import StudentForm
```

```
3) # Create your views here.
```

```
4) def student_input_view(request):
```

```
5)     sent=False
```

```
6)     if request.method=='POST':
```

```
7)         form=StudentForm(request.POST)
```

```
8)         if form.is_valid():
```

```
9)             print('Form Validation Success and printing data')
```

```
10)            print('Name:',form.cleaned_data['name'])
```

```
11)            print('Marks:',form.cleaned_data['marks'])
```

```
12)            sent=True
```

```
13)     form=StudentForm()

14)     return render(request,'testapp/input.html',{ 'form':form,'sent':sent})
```

input.html

1) <!DOCTYPE html>

```
2) <html lang="en" dir="ltr">
```

```
3)     <head>
```

```
4)         <meta charset="utf-8">
```

```
5)         <title></title>
```

```
6)         <!-- Latest compiled and minified CSS -->
```

```
7) <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css"
  integrity="sha384-BVYiiSIFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68vbdEjh4u"
  crossorigin="anonymous">
```

```
8)     </head>
```

```
9)     <body>
```

```
10)         <div class="container" align="center">
```

```
11)             {% if sent %}
```

```
12)                 <h1>Thanks for Providing Information</h1>
```

```
13)                 <h2>Enter Next Student Marks</h2>
```

```
14)             {% else %}
```

```
15)                 <h1>Student Marks Submit Form</h1>
```

16) {%endif%}<hr>

17) <form method='POST'>

18) {{form.as_p}}

19) {%csrf_token%}

20) <input type="submit" name="" class="btn btn-lg btn-
 primary' value="Submit Marks">

21) </form>

22) </div>

23) </body>

24) </html>

thankyou.html

1) <!DOCTYPE html>

2) <html lang="en" dir="ltr">

3) <head>

4) <meta charset="utf-8">

5) <title></title>

6) </head>

7) <body>

8) <h1>Thanks for providing your Name and Marks</h1>

9) </body>

10) `</html>`

urls.py

1) `from django.conf.urls import url`

2) `from django.contrib import admin`

3) `from testapp import views`

5) `urlpatterns = [`

6) `url(r'^admin/', admin.site.urls),`

7) `url(r'^input', views.student_input_view),`

Form Validations:

Once we submit the form we have to perform validations like

- 1) **Length of the field should not be empty**
- 2) **The max number of characters should be 10**
- 3) **The first character of the name should be 'd' etc**

We can implement validation logic by using the following 2 ways.

- 1) **Explicitly by the programmer by using clean methods**
- 2) **By using Django inbuilt validators**

Note: All validations should be implemented in the forms.py file

1} Explicitly by the Programmer by using Clean Methods:

- The syntax of clean method: `clean_fieldname(self)`
- In the FormClass for any field if we define clean method then at the time of submit the form, Django will call this method automatically to perform

validations. If the clean method won't raise any error then only form will be submitted.

```
from django import forms
```

```
class FeedBackForm(forms.Form):
```

```
    name=forms.CharField()
```

```
    rollno=forms.IntegerField()
```

```
    email=forms.EmailField()
```

```
    feedback=forms.CharField(widget=forms.Textarea)
```

```
    inputname=self.cleaned_data['name']
```

```
    if len(inputname) < 4:
```

```
        raise forms.ValidationError('The Minimum no of characters in the  
        name field should be 4')
```

```
    return inputname
```

The returned value of clean method will be considered by Django at the time of submitting the form.

[forms.py](#)

```
from django import forms
```

```
from django.core import validators
```

```
class FeedBackForm(forms.Form):
```

```
    name=forms.CharField()
```

```
6)    rollno=forms.IntegerField()
```

```
7)    email=forms.EmailField()
```

```
8)      feedback=forms.CharField(widget=forms.Textarea)
```

```
9)
```

```
10)     def clean_name(self):
```

```
11)         print('validating name')
```

```
12)         inputname=self.cleaned_data['name']
```

```
13)         if len(inputname) < 4:
```

```
14)             raise forms.ValidationError('The Minimum no of characters in  
the name field  
should be 4')
```

```
15)         return inputname+'Mahto'
```

```
16)     def clean_rollno(self):
```

```
17)         inputrollno=self.cleaned_data['rollno']
```

```
18)         print('Validating rollno field')
```

```
19)         return inputrollno
```

```
20)     def clean_email(self):
```

```
21)         inputemail=self.cleaned_data['email']
```

```
22)         print('Validating email field')
```

```
23)         return inputemail
```

```
24)     def clean_feedback(self):

25)         inputfeedback=self.cleaned_data['feedback']

26)         print('Validating feedback field')

27)         return inputfeedback
```

2) Django's Inbuilt Core Validators:

☕ Django provides several inbuilt core validators to perform very common validations.

☕ We can use these validators directly and we are not required to implement.

☕ Django's inbuilt validators are available in the `django.core` module.

☕ `from django.core import validators`

☕ To validate Max number of characters in the feedback as 40, we have to use inbuilt validators as follows.

forms.py

```
1) from django import forms

2) from django.core import validators

3)
```

```
4) class FeedBackForm(forms.Form):
```

```
5)     name=forms.CharField()
```

```
6)     rollNo=forms.IntegerField()
```

```
7)     email=forms.EmailField()
```

```
8)     feedback=forms.CharField(widget=forms.Textarea,validators=[  
        validators.MaxLengthValidator(40)])
```

Note: We can use any number of validators for the same field

```
feedback = forms.CharField(widget = forms.Textarea,validators =  
[validators.MaxLengthValidator(40),validators.MinLengthValidator(10)])
```

Note: Usage of built in validators is very easy when compared with clean methods.

How to implement Custom Validators by using the same Validator Parameter:

The value of name parameter should starts with 'd' or 'D'. We can implement this validation as follows

Validation of Total Form directly by using a Single Clean Method:

Whenever we are submitting the form Django will call clean() method present in our Form class. In that method we can implement all validations.

forms.py

```
1) from django import forms
```

```
2) from django.core import validators
```

```
3)
```

```
4) class FeedBackForm(forms.Form):
```

```
5)     name=forms.CharField()
```

```
6)     rollno=forms.IntegerField()
```

```
7)     email=forms.EmailField()
```

```
8)     feedback=forms.CharField(widget=forms.Textarea)
```

```
9)
```

```
10)    def clean(self):
```

```
11)        print('Total Form Validation...')
```

```
12)        total_cleaned_data=super().clean()
```

```
13)        inputname=total_cleaned_data['name']
```

```
14)        if inputname[0].lower() != 'd':
```

```
15)            raise forms.ValidationError('Name parameter should starts  
with d')
```

```
16)        inputrollno=total_cleaned_data['rollno']
```

```
17)        if inputrollno <=0:
```

```
18)            raise forms.ValidationError('Rollno should be > 0')
```

How to Check Original pwd and reentered pwd are same OR not in Signup Form:

forms.py

1) `from django import forms`

2) `from django.core import validators`

3)

4) `class FeedbackForm(forms.Form):`

5) `name=forms.CharField()`

6) `rollno=forms.IntegerField()`

7) `email=forms.EmailField()`

8) `feedback=forms.CharField(widget=forms.Textarea)`

9)

10) `def clean(self):`

11) `print('Total Form Validation by using single clean method...')`

12) `total_cleaned_data=super().clean()`

13) `inputname=total_cleaned_data['name']`

14) `if inputname[0].lower() != 'd':`

15) `raise forms.ValidationError('Name should starts with d|D')`

```
16)    inputrollno=total_cleaned_data['rollno']

17)    if inputrollno<=0:

18)    raise forms.ValidationError('Rollno should be >0')
```

```
19) class SignupForm(forms.Form):

20)     name=forms.CharField()

21)     password=forms.CharField(widget=forms.PasswordInput)

22)     rpassword=forms.CharField(label='Reenter
    Password',widget=forms.PasswordInput)

23)     email=forms.EmailField()

24)

25)     def clean(self):

26)         total_cleaned_data=super().clean()

27)         pwd=total_cleaned_data['password']

28)         rpwd=total_cleaned_data['rpassword']

29)         if pwd != rpwd:

30)             raise forms.ValidationError('Both Passwords must be same')
```

views.py

1)

```
from django.shortcuts import render
```

2)

```
from testapp.forms import
```

```
FeedbackForm,SignupForm
```

 3)

4)

```
# Create your views here.
```

5)

```
def feedback_form_view(request):
```

6)

```
    form=FeedbackForm()
```

7)

```
    # feedback_submitted=False
```

8)

```
    if request.method=='POST':
```

9)

```
        form=FeedbackForm(request.POST)
```

10)

```
        if form.is_valid():
```

11)

```
            print('Basic Validation Completed and Printing Data')
```

12)

```
            print('Name:',form.cleaned_data['name'])
```

13)

```
            print('RollNo:',form.cleaned_data['rollno'])
```

14)

```
            print('Email:',form.cleaned_data['email'])
```

15)

```
            print('Feedback:',form.cleaned_data['feedback']) #  
            feedback_submitted=True
```

16)

```
    return render(request,'testapp/feedback.html',{'form':form})
```

17)

18)

```
def signup_form_view(request):
```

19)

```
    form=SignupForm()
```

20)

```
    if request.method=='POST':
```

21)

```
        form=SignupForm(request.POST)
```

22)

```
        if form.is_valid():
```

23)

```
            print('Basic Validation Completed and Printing Data')
```

24)

```
            print('Name:',form.cleaned_data['name'])
```



```
25)         print('Password:',form.cleaned_data['password'])

26)         print('Email:',form.cleaned_data['email'])

27)     return render(request,'testapp/signup.html',{'form':form})
```

Note: If we want to perform validations by comparing multiple input fields then we should go for single clean() method.

How to prevent Requests from BOT:

Generally form requests can be send by end user. Sometimes we can write automated programming script which is responsible to fill the form and submit. This automated programming script is nothing but BOT.

The main objectives of BOT requests are:

- 1) To create unnecessary heavy traffic to the website, which may crash our application.
- 2) To spread malware (viruses)

Being web developer compulsory we have to think about BOT requests and we have to prevent these requests.

How to prevent BOT Requests:



In the form we will place one hidden form field, which is not visible to the end user.

Hence there is no chance of providing value to this hidden field.



But BOT will send the value for this hidden field also.If hidden field got some value means it is the request from BOT and prevent that form submission.

```
from django import forms
```

```
2) from django.core import validators
```

3)

4) `class FeedBackForm(forms.Form):`

5) `name=forms.CharField()`

6) `password=forms.CharField(widget=forms.PasswordInput)`

7) `rpassword=forms.CharField(label='Re Enter Password',
widget=forms.PasswordInput)`

8) `rollno=forms.IntegerField()`

9) `email=forms.EmailField()`

10) `feedback=forms.CharField(widget=forms.Textarea)`

11) `bot_handler=forms.CharField(required=False,widget=forms.HiddenInput)`

13) `def clean(self):`

14) `print('validating passwords match...')`

15) `total_cleaned_data=super().clean()`

16) `fpwd=total_cleaned_data['password']`

17) `spwd=total_cleaned_data['rpassword']`

18) `if fpwd != spwd:`

19) `raise forms.ValidationError('Both passwords must be matched')`

20) `bot_handler_value=total_cleaned_data['bot_handler']`

```
21) if len(bot_handler_value)>0:
```

```
22) raise forms.ValidationError('Request from BOT...cannot be submitted!!!')
```

Note: Other ways to prevent BOT requests

- 1) By using Captchas
- 2) By using image recognizers

(like choose images where car present, and all)

Working with Django Models Form

Model Forms (Forms based on Model):

☕ Sometimes we can create form based on Model, such type of forms are called model based forms or model forms.

☕ The main advantage of model forms is we can grab end user input and we can save that input data very easily to the database.

☕ Django provides inbuilt support to develop model based forms very easily.

How to develop Model based Forms:

1) While develop FormClass instead of inheriting forms.Form class, we have to inherit forms.ModelForm class.

```
class RegisterForm(forms.ModelForm):
```

```
....
```

2) We have to write one nested class (Meta class) to specify Model information and required fields.

```
class RegisterForm(forms.ModelForm):
```

```
    # field declarations if we are performing any custom validations. If we
    # are not defining any custom validations then here we are not required
    # to specify any field.
```

```
    class Meta:
```

```
        # we have to specify Model class name and required fields
        model=Student
```

```
        fields='_all_'
```

Case-1: Instead of all fields if we want only selected fields, then we have to specify as follows

```
class Meta:
```

```
    model=Student
    fields=('field1','field2','field3')
```

In the form only 3 fields will be considered.

If Model class contains huge number of fields and we required to consider very less number of fields in the form then we should use this approach.

Case-2:

Instead of all fields if we want to exclude certain fields, then we have to specify as follows:-

```
class Meta:
    model=Student
    exclude=['field1','field2']
```

In the form all fields will be considered except field1 and field2.

If the Model class contains huge number of fields and if we want to exclude very few fields then we have to use this approach.

Q) In Model based Forms, how many ways are there to specify Fields Information

Ans: 3 ways

- 1) All fields
- 2) Include certain fields
- 3) Exclude certain fields

Note: The most commonly used approach is to include all fields.

How to Save User's Input Data to Database in Model based Forms:

We have to use save() method.

```

def student_view(request):
    ..
    .
    if request.method=='POST':

        form=RegisterForm(request.POST)

        if form.is_valid():

            form.save(commit=True)

        ..

```

Now define ModelForm class as:

```

class MoviesForm(forms.ModelForm):

```

```

    movie_name=forms.CharField(widget=forms.TextInput(attrs={'class':'form-control'}))
    release_date=forms.DateField(widget=DateInput(attrs={'class':'form-control'}))

```

--> Here 'class':'form-control' is a bootstrap class used to format the date field

```

class Meta:

```

```

    model=MoviesModel

    fields=['movie_name','release_date']

```

Working With Advanced Template

Advanced Templates:

- 1) **Template Inheritance**
- 2) **Template Filters**
- 3) **Template tags for relative URLs**

1) Template Inheritance:

☕ If multiple template files have some common code, it is not recommended to write that common code in every template html file. It increases length of the code and reduces readability. It also increases development time.

☕ We have to separate that common code into a new template file, which is also known as base template. The remaining template files should be required to extend base template so that the common code will be inherited automatically.

☕ Inheriting common code from base template to remaining templates is nothing but template inheritance.

How to implement Template Inheritance:

base.html

1)

<!DOCTYPE html>

2) html, css, bootstrap links

3) <body>

4) common code required for every child template

5) {% block child_block %}

6) Anything outside of this block available to child tag.

7) in child template the specific code should be in this block

8) `{%endblock%}`

9) `</body>`

10) `</html>`

Advantages of Template Inheritance:

- 1) What ever code available in base template is by default available to child templates and we are not required to write again.Hence it promotes Code Reusability (DRY).
- 2) It reduces length of the code and improves readability.
- 3) It reduces development time.
- 4) It provides unique and same look and feel for total web application.

Note: Based on our requirement we can extend any number of base templates.i.e Multiple Inheritance is applicable for templates.

How to Add Seperate CSS Files to Child Templates?

base.html

1) `<!DOCTYPE html>`

2) `{%load static %}`

3) `<html lang="en" dir="ltr">`

4) `<head>`

5) `<meta charset="utf-8">`

6) `<title></title>`

7) `<link rel="stylesheet"`

`href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7`

`/css/bootstrap.min.css" integrity="sha384-`

`BVYiISiFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68`

`vbdEjh4u" crossorigin="anonymous">`

8) `<link rel="stylesheet" href="{% static "css/advtempot.css"%}">`

9) `{% block xyz %}{% endblock %}`

10) `</head>`

11) `<body>`

12) `<nav class="navbar navbar-default navbar-inverse">`

13) `<div class="container">`

14) `<!-- Brand and toggle get grouped for better mobile display -->`

15) `<div class="navbar-header">`

16) `<a class="navbar-brand" href="/">AJEET NEWS</a>`

17) `</div>`

18) `<ul class="nav navbar-nav">`

19) `<li><a href="/">Home</a></li>`

20) `<li><a href="/movies">Movies</a></li>`

21) `</li><a href="/sports">Sports</a></li>`

22) `<li><a href="/politics">Politics</a></li>`

23) `</ul>`

24) `</div><!-- /.container-fluid -->`

25) `</nav>`

26) `{% block body_block %}`

27) `{% endblock %}`

28) `</body>`

29) `</html>`

child.html

1) `<!DOCTYPE html>`

4) `{% block xyz %}`

3) `{%load staticfiles%}`

5) `<link rel="stylesheet" href="{%static "css/new.css" %}">`

8) `<h1>Welcome to DURGA NEWS PORTAL</h1>`

7) `{% block body_block %}`

9) {% endblock %}

2) Tempalte Filters:

☕ In the template file, the injected data can be displayed by using template tags.

☕ `{{emp.eno}}`

☕ Before displaying to the end user if we want to perform some modification to the injected text, like cut some information or converting to title case etc, then we should go for Template filters.

Syntax of Template Filter:

`{{value|filtername:"argument"}}` Filter may take or may not take arguments.i.e arguments are optional.

Eg:

`<li>{{msg1|lower}}</li>`

msg1 will be displayed in lower case

`<li>{{msg3|add:"Mahto"}}</li>`

"Durga" will be added to msg3 and then display the result to the end user

`{{ msg|title }}`

`{{ my_date|date:"Y-m-d" }}`

Note: There are tons of built in filters are available.

<https://docs.djangoproject.com/en/2.1/ref/templates/builtins/#ref-templates-builtins-filters>

How to Create our own Filter:

Based on our requirement we can create our own filter.

Steps:

- 1) Create a folder 'templatetags' inside our application folder.
- 2) Create a special file named with `_init_.py` inside templatetags folder, so that Django will consider this folder as a valid python package
- 3) Create a python file inside templatetags folder to define our own filters `cust_filters.py` -->any name

Note: We can also register filter with the decorator as follows.

1)

```
from django import template
```

2) `register=template.Library()`

3)

4) `@register.filter(name='f8upper')`

5) `def first_eight_upper(value):`

6) `"""This is my own filter"""`

7) `result=value[:8].upper()`

8) `return result`

f8upper is the name of the filter which can be used inside template file.

4) Inside template file we have to load the filter file as follows(In the child template but not in base template)

```
{%load cust_filters%
```

5) We can invoke the filter as follows `{{msg|f8upper}}`

Eg 2:

Custom Filter with argument

```
@register.filter(name='c_and_c')
def cut_and_concate(value,arg):
    result=value[:4]+str(arg) return result
```

Note: The main advantage of template filters is we can display the same data in different styles based on our requirement.

Example-2

cust_filters.py

1)

```
from django import template
```

2) `register=template.Library()`

3)

4) `@register.filter(name='truncate5')`

5) `def truncate5(value):`

6) `result=value[0:5]`

7) `return result`

8)

9) `@register.filter(name='t_n')`

10) `def truncate_n(value,n):`

11) `result=value[0:n]`

12) `return result`

lower.html

1) `<!DOCTYPE html>`

4) `<meta charset="utf-8">`

3) `<head>`

5) `<title></title>`

8) `{%load cust_filters%}`

7) `<body>`

9) `{% for r in data_list%`

12) `<li>{{r.subject|t_n:3}}</li>`

11) `<li>{{r.name|t_n:6}}</li>`

13) `<li>{{r.dept|t_n:2}}</li>`

16)
`</ul><hr>`

15) `<li>{{r.date|date:"d-m-Y"}}</li>`

17) `{%endfor%}`

19) `</html>`

Session Management

Session Management:

☕ Client and Server can communicate with some common language which is nothing but HTTP.

☕ The basic limitation of HTTP is, it is stateless protocol. i.e it is unable to remember client information for future purpose across multiple requests. Every request to the server is treated as new request.

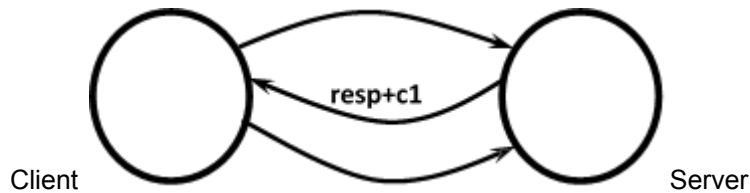
☕ Hence some mechanism must be required at server side to remember client information across multiple requests. This mechanism is nothing but session management mechanism.

☕ The following are various session management mechanisms.

- 1) Cookies
- 2) Session API
- 3) URL Rewriting
- 4) Hidden Form Fields etc

Session Management By using Cookies:

Cookie is a very small amount of information created by Server and maintained by client.



- Whenever client sends a request to the server, if server wants to remember client information for the future purpose then server will create cookie object with the required information.
- Server will send that Cookie object to the client as the part of response.
- Client will save that cookie in its local machine and send to the server with every consecutive request.
- By accessing cookies from the request server can remember client information.

Application-1 Session Management by using Cookies **(PageCount Application)**

[views.py](#)

```
1) from django.shortcuts import render
2)
3) # Create your views here.
4) def count_view(request):
5)     if 'count' in request.COOKIES:
6)         newcount=int(request.COOKIES['count'])+1
7)     else:
8)         newcount=1
9)     response=render(request,'testapp/count.html',{'count':newcount})
10)    response.set_cookie('count',newcount)
11)    return response
```

count.html

```
1) <!DOCTYPE html>
```

```
4)    <meta charset="utf-8">
```

```
3)    <head>
```

```
5)    <title></title>
```

```
7)    span{
```

```
8)        font-size: 200px;
```

```
9)        font-weight: 900;
```

```
10)     }
```

```
11)
```

```
12)    </style>
```

```
13) </head>
```

14) `<body>`

15) `<h1>Page Count is: <span> {{count}}</span></h1>`

16) `</body>`

17) `</html>`

home.html

1) `<!DOCTYPE html>`

4) `<meta charset="utf-8">`

3) `<head>`

5) `<title></title>`

7) `<body>`

8) `<h1>Welcome to PYSPIDER</h1>`

9) `<form action="/second">`

10) `{{form.as_p}}`

11) `{%csrf_token%}`

12) `<input type="submit" name="" value="Enter Name">`

13) `</form>`

14) `</body>`

15) `</html>`

datetime.html

1) `<!DOCTYPE html>`

3) `<head>`

6)
`</head>`

5) `<title></title>`

7) `<body>`

10) `</body>`

9) `<a href="/result">Click Here to get Date and Time</a>`

11) `</html>`

result.html

1) `<!DOCTYPE html>`

2) `<html lang="en" dir="ltr">`

3) `<head>`

4) `<meta charset="utf-8">`

5) `<title></title>`

6) `</head>`

7) `<body>`

8) `<h1>Hello {{name}}</h1><hr>`

9) `<h1>Current Date and Time:{{date_time}}</h1>`

10) `<a href="/result">Click Here to get Updated Date and Time</a>`

11) `</body>`

12) `</html>`

Application-3 Session Management by using Cookies

views.py

1) `from django.shortcuts import render`

2)

3) `# Create your views here.`

4) `def name_view(request):`

5) `return render(request, 'testapp/name.html')`

6)

7) **def** age_view(request):

8) name=request.GET['name']

9) response=render(request,'testapp/age.html',{'name':name})

10) response.set_cookie('name',name)

11) **return** response

12)

13) **def** gf_view(request):

14) age=request.GET['age']

15) name=request.COOKIES['name']

16) response=render(request,'testapp/gf.html',{'name':name})

17) response.set_cookie('age',age)

18) **return** response

19)

20) **def** results_view(request):

21) name=request.COOKIES['name']

22) age=request.COOKIES['age']

23) `gfname=request.GET['gfname']`

24) `response=render(request,'testapp/results.html',{'name':name,'age':age,'gfname':gfname})`

25) `response.set_cookie('gfname',gfname)`

26) `return response`

name.html

1) `<!DOCTYPE html>`

4) `<meta charset="utf-8">`

3) `<head>`

5) `<title></title>`

8) `<h1>Welcome to App</h1>`

7) `<body>`

9) `<form action="/age">`

11) `<input type="submit" name="" value="Submit Name">`

12)

13)

`</body>`

`</form>`

14) `</html>`

age.html

1) `<!DOCTYPE html>`

2) `<html lang="en" dir="ltr">`

3) `<head>`

4) `<meta charset="utf-8">`

5) `<title></title>`

6) `</head>`

7) `<body>`

8) `<h1>Hello {{name}}..</h1><hr>`

9) `<form action="/gf">`

10) Enter Age: `<input type="text" name="age" value=""><br><br>`

11) `<input type="submit" name="" value="Submit Age">`

12) `</form>`

13) `</body>`

14) `</html>`

gf.html

1) `<!DOCTYPE html>`

2) `<html lang="en" dir="ltr">`

3) `<head>`

4) `<meta charset="utf-8">`

5) `<title></title>`

6) `</head>`

7) `<body>`

8) `<h1>Hello {{name}}.</h1><hr>`

9) `<form action="/results">`

9)

Enter Girl Friend Name: `<input type="text" name="gfname" value=""><br><br>`

11)

14)

`</html>`

`</body>`

results.html

1) `<!DOCTYPE html>`

2) `<html lang="en" dir="ltr">`

3) `<head>`

4) `<meta charset="utf-8">`

5) `<title></title>`

6) `</head>`

7) `<body>`

8) `<h1>Hello {{name}} Thanks for providing info</h1>`

9) `<h2>Please cross check your data and confirm</h2><hr>`

10) `<ul>`

11) `<li>Name:{{name}}</li>`

12) `<li>Age:{{age}}</li>`

13) `<li>Girl Friend Name:{{gfname}}</li>`

14) `</ul>`

15) `</body>`

16) `</html>`

Limitations of Cookies:

- 1) By using cookies we can store very less amount of information. The size of the cookie is fixed. Hence if we want to store huge amount of information then cookie is not best choice.
- 2) Cookie can hold only string information. If we want to store non-string objects then we should not use cookies.
- 3) Cookie information is stored at client side and hence there is no security.
- 4) Everytime with every request, browser will send all cookies related to that application, which creates network traffic problems.
- 5) There is a limit on the max number of cookies supported by browser.

To overcome all these limitations we should go for sessions.

Temporary vs Permanent Cookies:

☕ If we are not setting any `max_age` for the cookie, then the cookies will be stored in browser's cache. Once we close the browser, the cookies will be expired. Such type of cookies are called temporary cookies.

☕ We can set temporary cookies as follows: `response.set_cookie(name, value)`

☕ If we are setting `max_age` for the cookie, then cookies will be stored in the local file system permanently. Once the specified `max_age` expires, then only cookies will be expired. Such type of cookies are called permanent or persistent cookies. We can set permanent cookies as follows:

```
response.set_cookie(name, value, max_age=180)
response.set_cookie(name, value, 180)
```

☕ The time unit for `max_age` is in seconds.

Application-4 Session Management by using Cookies

(Shopping Cart Application)

views.py

```
from django.shortcuts import render
```

```
def index(request):
```

```
    return render(request, 'testapp/home.html')
```

```
def additem(request):
```

```
    form=ItemAddForm()
```

```

        response=render(request,'testapp/additem.html',{'form':form})

if request.method=='POST':

form=ItemAddForm(request.POST)

if form.is_valid():

    name=form.cleaned_data['itemname']

    quantity=form.cleaned_data['quantity']

    response.set_cookie(name,quantity,180)

    return response

def displayitem_view(request):

    return render(request,'testapp/showitems.html')

```

Session Management By using Session API (Django Session Framework):

☕ Once client sends request to the server, if server wants to remember client information for the future purpose then server will create session object and store required information in that object. For every session object a unique identifier is available which is nothing but sessionid.

☕ Server sends the corresponding session id to the client as the part of response. Client retrieves the session id from the response and save in the local file system. With every consecutive request client will that session id. By accessing that session id and corresponding session object server can remember client. This mechanism is nothing but session management by using session api.

Note: Session information will be stored in one of following possibilities

1. Inside a File

2. Inside a database

3. Inside Cache

The most straight forward approach is to use `django.contrib.sessions` application to store session information in a Django Model/database.

The Model Name is: `django.contrib.sessions.models.Session`

Note: To use this approach compulsory the following applicaiton should be configured inside `INSTALLED_APPS` list of `settings.py` file.

`django.contrib.sessions`

If it is not there then we can add, but we have to
synchronize database `python manage.py syncdb`

Note:

`INSTALLED_APPS = [`

`....`

`'django.contrib.sessions',`

`...`

`]`

`MIDDLEWARE = [`

`..`

`'django.contrib.sessions.middleware.SessionMiddleware',`

`....`

`]`

Useful Methods for Session Management:

1)

`request.session['key'] = value`
To Add Data to the Session.

2)

`value = request.session['key']`
To get Data from the Session

3)

`request.session.set_expiry(seconds)` Sets the expiry Time for the Session.

4) `request.session.get_expiry_age()`

Returns the expiry age in seconds(the number of seconds until this session expire)

5) `request.session.get_expiry_date()`

Returns the data on which this session will expire

Note: Before using session object in our application, compulsory we have to migrate. Otherwise we will get the following error.

no such table: django_session

Application-1 Session Management by using Session API (PageCount Application)

[views.py](#)

2)

3) # Create your views here.

4) `def page_count_view(request):`

5) `count=request.session.get('count',0)`

6) `newcount=count+1`

7) `request.session['count']=newcount`

8) `print(request.session.get_expiry_age())`

9) `print(request.session.get_expiry_date())`

10) `return`

`render(request,'testapp/pagecount.html',{'count':newcount})`

Pagecount.html

1) `<!DOCTYPE html>`

4)

`<meta charset="utf-8">`

3) `<head>`

5) `<title></title>`

6) `<style >`

7) `span{`

8) `font-size: 300px;`

9) `}`

10) `</style>`

11) `</head>`

12) `<body>`

13) `<h1>The Page Count:<span>{{count}}</span></h1>`

14) `</body>`

15) `</html>`

Shopping cart application in by using the session

Note: The default max_age of the session is 14 days. But based on our requirement we can set our own expiry time.

```
request.session.set_expiry(time in seconds)
```

Important Methods related to Session:

1) set_expiry(time in seconds)

- If we are not performing any operation on the session specified amount of time (max inactive interval) then session will be expired automatically.

```
request.session.set_expiry(120)
```

- If we set 0 as argument, then the session will expire once we closed browser.

2) `get_expiry_age()`

3) `get_expiry_date()`

Note: We can observe that 0 value and 120 values are perfectly working in our application

How to Delete Session Data:

```
del request.session[sessionkey]
```

```
for key in  
    request.session.keys():  
    del  
        request.session[key]
```

In settings.py File

```
SESSION_SAVE_EVERY_REQUEST=True
```

OR

In views.py

```
request.session.modified = True
```

Browser Length Sessions and Persistent Sessions

☕ If the session information stored inside browser's cache such type of sessions are called browser length sessions.

☕ If the session information stored persistently inside file/database/cache, such type of sessions are called Persistent sessions.

Note: By default sessions are persistent sessions.

Chapter-9

User

Authentication

And

Authorization

User Authentication and Authorization

Authentication: The process of validating user is called authentication.

Authorization: The process of validating access permissions of user is called authorization.

Generally our web pages can be accessed by any person without having any restrictions. But some times our business requirement is to access a web page compulsory we have to register and login. Then only end user can be able to access our page. To fulfill such of requirements we should go for Django authentication and authorization module.

(auth application)

Django provides the following 2 in built applications for user authentication.

- 1) `django.contrib.auth`
- 2) `django.contrib.contenttypes`

auth application is authentication application of Django.

This auth application internally uses contenttypes application to track models installed in our database.

Note: To use Django in built authentication facility, compulsory these 2 applications should be in `INSTALLED_APPS` list. But from Django 1.10 onwards automatically these are available and we are not required to add explicitly.

Django uses `PBKDF2_Sha256` algorithm to encrypt passwords and hence passwords won't be stored in plain text form and we can expect more security. Even superuser also cannot see any user's password.

Based on our requirement, we can use more secure hashing algorithms also like `bcrypt` and `argon2`. We can install with pip as follows.

pip install bcrypt

pip install django[argon2]

More secured algorithm is argon2 and then bcrypt

followed PBKDF2. In settings.py we have to configure

password hashers as follows.

```
1) PASSWORD_HASHERS=[  
  
4)'django.contrib.auth.hashers.BCryptPasswordHasher',
```

```
3)'django.contrib.auth.hashers.BCryptSHA256PasswordHasher',
```

```
5)'django.contrib.auth.hashers.PBKDF2PasswordHasher',
```

```
6)'django.contrib.auth.hashers.PBKDF2SHA1PasswordHasher',]
```

Django will always consider from first to last. ie order is important.

Just like templates and static folder, we have to create media folder also.

Difference between Static and Media Folders:

☕ **Static folder contains images, CSS files etc which are provided by application to the end user.**

☕ **But media folder contains the resources like images provided by end user to the application (like profile image etc)**

How to Configure Media Folder in settings.py File:

```
MEDIA_DIR =  
os.path.join(BASE_DIR,'media')  
MEDIA_ROOT = MEDIA_DIR  
  
MEDIA_URL = '/media/'
```

```
from django.contrib.auth.decorators import
```

```
login_required @login_required
```

```
def java_exams_view(request):
```

```
    return render(request, 'testapp/java.html')
```

If we use `@login_required` decorator for any view function, then auth application will check whether user login or not. If the user not login then the control will be forwarded to login page.

<http://127.0.0.1:8000/accounts/login/?next=/java/>

We have to configure auth application url patterns in

urls.py file. `from django.conf.urls import url, include`

```
url('accounts/',
```

```
include('django.contrib.auth.urls')), In our
```

project auth application urls also

included.

TemplateDoesNotExist at

`/accounts/login/registration/login.html` login link of auth

application: `/accounts/login/`

logout link of auth application: `/accounts/logout`

After logout then Django default logout page will be displayed. Instead of this default target page we can configure our own target page inside `settings.py` as follows.

LOGOUT_REDIRECT_URL='/'

If we click login link explicitly and after login by default the control will goes to <http://127.0.0.1:8000/accounts/profile/>

But we can configure our own target page after login inside settings.py as follows.

LOGIN_REDIRECT_URL='/'

Authentication Application:

views.py

from django.shortcuts import render

def home_page_view(request):

def java_exams_view(request):

@login_required

13) return render(request,'testapp/pythonexams.html')

15) def aptitude_exams_view(request):

17)

19) return render(request,'testapp/logout.html')

urls.py

1) from django.conf.urls import url,include

2) from django.contrib import admin

3) from testapp import views

4)

5) urlpatterns = [

6) url(r'^admin/', admin.site.urls),

7) url('accounts/',include('django.contrib.auth.urls')),

8) url(r'^\$', views.home_page_view),

9) url(r'^python/', views.python_exams_view),

10) url(r'^java/', views.java_exams_view),

11) url(r'^aptitude/', views.aptitude_exams_view),

13)]

auth123.css

1) body{

2) background: #efb917;

3) color:blue;

}

5) .jumbotron{

6) background: green;

7) color:white;

8) }

base.html

- 1) <!DOCTYPE html>
- 2) {%load staticfiles%}
- 3) <html lang="en" dir="ltr">
- 4) <head>
- 5) <meta charset="utf-8">
- 6) <title></title>
- 7) <!-- Latest compiled and minified CSS -->
- 8) <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/cs
s/bootstrap.min.css" integrity="sha384-
BVYiISiFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTs
z/K68vbdEjh4u" crossorigin="anonymous">
- 9) <link rel="stylesheet" href="{%static "css/auth123.css"%}">
- 10) </head>
- 11) <body>
- 12) <nav class='navbar navbar-default navbar-inverse'>
- 13) <div class="container">
- 14) <div class="navbar-header">
- 15) DURGAEXAMS
- 16) </div>
- 17) <ul class='nav navbar-nav'>

18) Java Exams

19) Python Exams

20) Aptitude Exams

21)

22) <ul class='nav navbar-nav navbar-right'>

23) Signup

24) Login

25) Logout

26)

27) </div>

28) </nav>

29) {%block body_block%}

30) {%endblock%}

31) </body>

32) </html>

Home.html

registration/login.html

- 1) <!DOCTYPE html>
- 2) {%extends 'testapp/base.html'%}
- 3) {%block body_block%}
- 4) <div class="container" align='center'>
- 5) <h1>Login to write Exams...</h1>
- 6) <form method="post">
- 7) {{form.as_p}}
- 8) {%csrf_token%}
- 9) <button type="submit">Login</button>
- 10) </form>
- 11) </div>
- 12) {%endblock%}

testapp/logout.html

- 5) <div class="container">
- 6) <h1>Thanks for visiting Durgaexams</h1>

7) <p>Anyway we are feeling very sad b'z you logout atleast 24 hours per day

you have to spend with our website like facebook</p>

8) <h2>Please Login Again:</h2>

9)

Login

10) </div>

11) </div>

12) {%endblock%}

settings.py

Configure Signup Form:

forms.py

from django import forms

from django.contrib.auth.models import User

class SignUpForm(forms.ModelForm):

class Meta:

 model=User

fields=['username','password','email','first_name','last_name']

views.py

urls.py

urlpatterns = [

url(r'^signup/', views.signup_view),

]

Password Hashers in settings.py

PASSWORD_HASHERS=[

'django.contrib.auth.hashers.BCryptSHA256PasswordHasher',

'django.contrib.auth.hashers.PBKDF2PasswordHasher',

]

Future Assignments:

- 1. How to customize our own login form**
- 2. How to use auth application provided default signup form**
- 3. Social Login**

Chapter-10

Class Based

Views And

CRUD

Operations By

Using Both

CBVs

And FBVs

Class Based Views (CBVs):

There are 2 types of views

- 1) **Function Based Views**
- 2) **Class Based Views**

Note:

- 1) Class Based Views introduced in Django 1.3 Version to implement Generic Views.
- 2) When compared with Function Based views, class Based views are very easy to use. Hence Class Based Views are Most frequently used views in real time.
- 3) Internally Class Based Views will be converted into Function Based Views. Hence Class Based Views simply act as wrappers to the Function based views to hide complexity.
- 4) Function Based views are more powerful when compared with Class Based Views.

Q) Explain the Scenario where we should use Function based Views only and we cannot use Class based Views?

☕ For simple operations like listing of all records or display details of a particular record then we should go for Class Based Views.

☕ For complex operations like handling multiple forms simultaneously then we should go for Function Based Views.

Eg: KFC

HelloWorld Application By using ClassBasedViews

views.py

- 1) `from django.views.generic import View`
- 3)


```
5) class HelloWorldView(View):  
7) return HttpResponse('<h1>This is from ClassBasedView</h1>')
```

urls.py

```
1) from testapp import views  
2)  
3) urlpatterns = [  
4)     ...  
5)     url(r'^$', views.HelloWorldView.as_view()),  
6) ]
```

Note:

- 1) While defining Class Based Views we have to extend View class.
- 2) To provide response to GET request, Django will always call get() method. Hence we have to override this method to provide response to the GET request. Similarly for other HTTP Methods also like POST, HEAD etc
- 3) While defining url pattern we have to use as_view() method.

Template Based Application by using Class Based Views:

```
class  
    TemplateCBV(Templa  
teView)  
    template_name =  
    'home.html'
```

How to send Context Parameters:

```
class  
    TemplateCBV(Templa  
teView)
```

```
template_name =  
'home.html'
```

```
def get_context_data(self,**kwargs)  
    context = super().get_context_data(**kwargs)  
    context['name'] = 'ajeet'  
    context['age'] =  
    30 return  
    context
```

In template file we can access context parameters as follows:

```
{{name}}
```

```
{{age}}
```

ListView:

We can use ListView class to list out all records from database table(model). It is alternative to `ModelClassName.objects.all()`

models.py

models.py

1) from django.db import models

2)

3) # Create your models here.

4) class Book(models.Model):

5) title=models.CharField(max_length=300)

6) author=models.CharField(max_length=30)

7) pages=models.IntegerField()

8) price=models.FloatField()

views.py

- 1) from testapp.models import Book
- 2) from django.views.generic import ListView
- 3)
- 4) # Create your views here.
- 5) class BookListView(ListView):
- 6) model=Book

How to create Template File for ListView:

Django will identify template automatically and we are not required to configure anywhere. But Django will always search for template file with the name `modelclassname_list.html` like `book_list.html`

Django will always search for template file in the following location.
`projectname/appname/templates/appname/`

Eg: `cbvproject5/testapp/templates/test/book_list.html`

Note: bydefault django will provide context object to the template file with the name: `modelclassname_list`

Eg: `book_list`

`book_list.html`

- 1) `<!DOCTYPE html>`
- 3) `<head>`
- 5) `<title></title>`

```
7)    <body>

9)    {% for book in book_list%}

11)   <li> <strong>Title</strong>:{{book.title}}</li>

13)   <li> <strong>Pages</strong>:{{book.pages}}</li>

15)   </ul>

17)   {%endfor%}

19) </html>
```

How to provide our own Context Object Name:

The default context object name is: modelclassname_list

But we can customize this name based on our requirement as follows

```
class BookListView(ListView):
    context_object_name = 'books'
    model = Book
```

How to configure our own Template File at Project Level:

ofcourse this approach is not recommended

```
class BookListView(ListViw):
    context_object_name = 'books'
    model = Book
    template_name = 'testapp/ajeet.html'
```

Note: Even if we are not specifying template_name variable, still django can recognize project level template file. But name should be modelclassname_list.html

DetailView:

We can use ListView to list of all records present in the database table. But to get details of a particular record, we should go for DetailView.

models.py

2)

3) `class Company(models.Model):`

4) `name=models.CharField(max_length=128)`

5) `location=models.CharField(max_length=64)`

6) `ceo=models.CharField(max_length=64)`

admin.py

1) `from django.contrib import admin`

2) `from testapp.models import Company`

3)

4) `# Register your models here.`

5) `class CompanyAdmin(admin.ModelAdmin):`

6) `list_display=['name','location','ceo']`

7)

8) `admin.site.register(Company,CompanyAdmin)`

`views.py`

1) `from django.shortcuts import render`

2) `from testapp.models import Company`

3) `from django.views.generic import ListView,DetailView`

4)

5) `# Create your views here.`

6) `class CompanyListView(ListView):`

7) `model=Company`

8) `#default template_name is company_list.html`

9) `#default context_object_name is company_list`

10)

11) `class CompanyDetailView(DetailView):`

12) `model=Company`

13) `#default template_name is company_detail.html`

14) `#default context_object_name is company or object`

`base.html`

1) `<!DOCTYPE html>`

3) `<head>`

5) `<title></title>`

7) `<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css" integrity="sha384-`

BVYiSiFeK1dGmJRAkycuHAHRg32OmUcww7on3RYdg4Va+PmSTsz/K68vbdEjh4u" c
rossorigin="anonymous">

9) <body>

11) {%block body_block%}

13) </div>

15) </html>

company_list.html

6) {%for company in company_list%}

7) <h2> {{company.name|upper}} </h2>

8) {%endfor%}

9)

10) {%endblock %}

company_detail.html

1) <!DOCTYPE html>

2) {%extends 'testapp/base.html'%}

3) {%block body_block%}

4) <h1>Company Information</h1><hr>

5)

6) <h2>Company Name: {{company.name|upper}}</h2>

7) <h2>Company Location: {{company.location|title}}</h2>

8) <h2>Company CEO: {{company.ceo|title}}</h2>

9)

10) {%endblock %}

urls.py

```
from django.conf.urls import url

from django.contrib import admin

from testapp import views

urlpatterns = [

    url(r'^admin/', admin.site.urls),

    url(r'^companies/', views.CompanyListView.as_view()),

    url(r'^int:<pk>/', views.CompanyDetailView.as_view()),

]
```

Adding Employee Information also

models.py

```
from django.db import models

# Create your models here.

name=models.CharField(max_length=128)

ceo=models.CharField(max_length=64)

location=models.CharField(max_length=64)

def str (self):

    return self.name

class Employee(models.Model):

    eno=models.IntegerField()
```


name=models.CharField(max_length=128)

salary=models.FloatField()

company=models.ForeignKey(Company,related_name='employees')

Django CRUD Operations

- 1) Create (Insert Query)
- 2) Retrieve (Select Query)
- 3) Update (Update Query)
- 4) Delete (Delete Query)

For any web application,it is very common requirement to perform CRUD operations.

Case Study: BookMyshow Application

- 1) Add New Movie Information (Create)
- 2) Show Movie Information (Retrieve)
- 3) Update New timings for existing Movie (Update)
- 4) Delete old Movie Information (Delete)

ListView, DetailView Retrieve Operation CreateView Create Operation (Insert Data) UpdateView Update Operation DeleteView Delete Operation

CreateView Class:

We can use this CreateView class to insert data into our models.

[views.py](#)

```
class CompanyCreateView(CreateView):
```

```
    model=Company
```

urls.py

```
1)    urlpatterns = [  
2)    url(r'^admin/', admin.site.urls),  
3)    url(r'^companies/', views.CompanyListView.as_view()),  
4)  
    url('/<pk>/',views.CompanyDetailView.as_view(),name='detail'),  
5)  
    url('create/',views.CompanyCreateView.as_view(),name='create'  
    '),  
]
```

If we send Request:

ImproperlyConfigured at /create/

Using ModelFormMixin (base class of CompanyCreateView) without the 'fields' attribute is prohibited.

We can solve this problem by defining fields attribute in

CreateView class

```
class CompanyCreateView(CreateView):
```

```
    model = Company
```

```
    fields = ('name','location','ceo')
```

If we send Request:

TemplateDoesNotExist at /create/ testapp/company_form.html

By default CreateView class will always search for template file named with modelclassname_form.html

Eg: company_form.html

company_form.html

1) <!DOCTYPE html>

2) {%extends 'testapp/base.html'%}

3) {%block body_block%}

4) <h1>Company Create Form</h1><hr>

5) <form method="post">

6) {{form.as_p}}

7) {%csrf_token%}

8) <input type="submit" class='btn btn-primary btn-lg' value="Insert Record">

9) </form>

10) {%endblock %}

If we fill form and submit:

ImproperlyConfigured at /create/

No URL to redirect to. Either provide a URL or define a `get_absolute_url` method on the Model.

How to define `get_absolute_url()` in Model class:

1) from django.db import models

3)

5) class Company(models.Model):

7) location=models.CharField(max_length=64)

9) def str (self):

12) return reverse('detail',kwargs={'pk':self.pk})

UpdateView class:

We can use UpdateView to update existing record.

views.py

1) class CompanyUpdateView(UpdateView):

3) fields=('name','ceo')

urls.py

Define URL for this updateview <http://localhost:8000/update/7>

Differences between CBVs and FBVs

CBVs	FBVs
1) CBVs can be easily extended	1) FBVs cannot extended easily
2) CBVs promotes Reusability of the Code	2) FBVs cannot promote Reusability of the Code
3) CBVs can use Object Oriented Techniques such as Mixins (Multiple Inheritance)	3) FBVs cannot use Object Oriented Techniques
4) In CBVs, Less Coding	4) In FBVs, More Coding
5) Default Context Dictionary and Default Template Files Support Available	5) Default Context Dictionary and Default Template Files Support not Available
6) Handling HTTP Methods by seperate Class Methods such as get() and post()	6) Handling HTTP Methods via Conditional Braching if <code>request.method == 'POST'</code>
7) There is a Restriction on Functionality and hence Less Power.	7) Based on Requirement we can implement any Functionality and hence these are more Powerful
8) Implicit Execution Flow and hence reduces Readability.	8) Explicit Execution Flow and hence improves Readability.

Note: In Real Time the most commonly used views are CBVs.If CBV can not handle our requirement then only we should go for FBVs.

Django ORM

Django ORM:

ORM Object Relation Mapping

In general we can retrieve data from the database by using the following approach

```
Employee.objects.all()
```

The return type of all() method is QuerySet. `qs=Employee.objects.all()`

```
print(type(qs)) # <class 'django.db.models.query.QuerySet'>
```

If we want to get only one record then we can use get()

```
method. emp = Employee.objects.get(id=1)
```

```
print(type(emp)) #<class 'testapp.models.Employee'>
```

The return type of get() method is Employee Object.

Case-1: How to find Query associated with QuerySet

```
qs = Employee.objects.all() print(qs.query)
```

Case-2: How to Filter Records based on some Condition

```
employees = Employee.objects.filter(esal gt=15000)
```

It returns all employees whose salary greater than 15000

```
employees = Employee.objects.filter(esal gte=15000)
```

It returns all employees whose salary greater than or equal to 15000 Similarly we can use lt and lte

Various possible Field Look ups:

1) exact Exact Match

Entry.objects.get(id exact=14)

2) iexact Case-insensitive exact Match Blog.objects.get(name iexact='beatles blog')
Blog.objects.get(name iexact=None)

The equivalent queries are:

SELECT ... WHERE name ILIKE 'beatles blog'; SELECT ... WHERE name IS NULL;

3) contains Case-sensitive Containment Test

Entry.objects.get(headline contains='Lennon')

SQL equivalent:

SELECT ... WHERE headline LIKE '%Lennon%';

4) icontains Case-insensitive Containment Test

Example: Entry.objects.get(headline icontains='Lennon')

5) in

In a given iterable; often a List, Tuple OR queryset.

Examples:

Entry.objects.filter(id in=[1, 3, 4]) Entry.objects.filter(headline in='abc')

6) gt

Greater than

Example: Entry.objects.filter(id gt=4)

7) gte

Greater than or equal to.

8) lt

Less than.

9) lte

Less than or equal to.

10) `startswith`

Case-sensitive starts-with.

Example: `Entry.objects.filter(headline startswith='Lennon')`

11) `istartswith`

Case-insensitive starts-with.

Example: `Entry.objects.filter(headline istartswith='Lennon')`

12) `endswith`

Case-sensitive ends-with.

Example: `Entry.objects.filter(headline endswith='Lennon')`

1) `iendswith`

Case-insensitive ends-with.

Example: `Entry.objects.filter(headline_iendswith='Lennon')`

2) `range`

Range test (inclusive)

1) `import datetime`

```
4) Entry.objects.filter(pub_date_range=(start_date,
end_date))
```

3) `end_date = datetime.date(2005, 3, 31)`

Eg-2: `employees=Employee.objects.filter(esal range=(12000,16000))`

Note: There are several other field lookups are possible.

(Documentation)

<https://docs.djangoproject.com/en/2.1/ref/models/queries/#id4>

Case-3: How to implement OR Queries in Django ORM

2 ways are available

- 1) `queryset_1 | queryset_2`
- 2) `filter(Q(condition1)|Q(condition2))`

Eg 1: To get all employees whose name startswith 'A' OR salary < 15000

```
employees = Employee.objects.filter(ename__startswith='A') |  
Employee.objects.filter(esal__lt=15000)  
from django.db.models import Q  
employees= Employee.objects.filter(Q(ename__startswith='A') |  
Q(esal__lt=15000))
```

Case-4: How to implement AND Queries in Django ORM

3 ways

- 1) `filter(condition1,condition2)`
- 2) `queryset_1 & queryset_2`
- 3) `filter(Q(condition_1)&Q(condition_2))`

Eg: To get all employees whose name startswith 'J' AND salary < 15000

- 1) `employees= Employee.objects.filter(ename__startswith='J',esal__lt=15000)`
- 2) `employees= Employee.objects.filter(ename__startswith='J') & Employee.objects.filter(esal__lt=15000)`

3) employees= Employee.objects.filter(Q(ename_startswith='J') & Q(esal
lt=15000))

Case-5: How to implement NOT Queries in Django ORM

2 ways

- 1) `exclude(condition)`
- 2) `filter(~Q(condition))`

Eg: To select all employees whose name not starts with 'J':

Case-6: How to perform Union Operation for Query Sets of the Same OR different Models

By using union operation, we can combine results of 2 or more query sets.

```
q1=Employee.objects.filter(esal____  
lt<15000)  
q2=Employee.objects.filter(ename____  
endswith='J') q3=q1.union(q2)
```

Note: The union operator can be performed only with the querysets having the same fields and data types. Otherwise we will get error saying `django.db.utils.OperationalError: SELECTs to the left and right of UNION do not have the same number of result columns.`

We can perform union operation on common columns.

Eg: Student(name, mailid, aadharnumber,
marks) Teacher(name, mailid,
aadharnumber, subject, salary)

```
q1 = Student.objects.all().values_list('name','mailid','aadha  
rnumber')  
q2 = Teacher.objects.all().values_list('name','mailid','aadha  
rnumber')  
q3 = q1.union(q2)
```

Case-7: How to select only some columns in the queryset

3 Ways

1) By using values_list()

Eg: q1 = Student.objects.all().values_list('name','mailid','aadharnumber')

2) By using values()

Eg: q1 = Student.objects.all().values('name','mailid','aadharnumber')

1) By using only():

Eg: q1 = Student.objects.all().only('name','mailid','aadharnumber')

Note: Difference between values() and only() Methods

In the case of values() only specified columns will be selected.
But in the case of only() in addition to specified columns 'id'
column also will be selected.

Case-8: Aggregate Functions

Django ORM defines several functions to perform aggregate operations. Avg(), Max(), Min(), Sum(), Count()

Working with Advanced Model Concepts

Kmk

Model Inheritance

It is very useful and powerful feature of django.

There are 4 types of Model Inheritance.

- 1) **Abstract Base Class Model Inheritance**
- 2) **Proxy Model Inheritance**
- 3) **Multiple Inheritance**

1) Abstract Base Class Model Inheritance

- If several Model classes having common fields, then it is not recommended to write these fields separately in every Model class. It increases length of the code and reduces readability.
- We can separate these common fields into another Model class, which is also known as Base Class. If we extend Base class automatically common fields will be inherited to the child classes.

Without Inheritance

- ```
1) class Student(models.Model):

3) email=models.EmailField()

5) rollno=models.IntegerField()

9) name=models.CharField(max_length=64)

11) address=models.CharField(max_length=256)

13) salary=models.FloatField()
```

#### **With Inheritance**

- ```
1) class ContactInfo(models.Model):  
  
2)     name=models.CharField(max_length=64)  
  
3)     email=models.EmailField()  
  
4)     address=models.CharField(max_length=256)  
  
5)     class Meta:  
  
6)         abstract=True  
  
7)  
  
8) class Student(ContactInfo):
```

`rollno=models.IntegerField()`

10) `marks=models.IntegerField()`

11)

12) `class Teacher(ContactInfo):`

13) `subject=models.CharField(max_length=64)`

14) `salary=models.FloatField()`

In this case only Student and Teacher tables will be created which includes all the fields of ContactInfo.

Note: ContactInfo class is an abstract class and hence table won't be created.

It is not possible to register abstract model classes to the admin interface.If we are trying to do then we will get error.

1) Multi Level Inheritance:

Inheritance at multiple levels.

`class Person(models.Model):`

`name=models.CharField(max_length=64)`

`age=models.IntegerField()`

`class Employee(Person):`

`eno=models.IntegerField()`

`esal=models.FloatField()`

`class Manager(Employee):`

`exp=models.IntegerField()`

`team_size=models.IntegerField()`

Note: Multilevel inheritance internally multitable inheritance only.

1) Multiple Inheritance:

If model class extends multiple parent classes simultaneously then such type of inheritance is called Multiple Inheritance.

```
class Parent1(models.Model):
```

```
    f2=models.CharField(max_length=64)
```

```
    f2=models.CharField(max_length=64)
```

```
class Parent2(models.Model):
```

```
    f4=models.CharField(max_length=64)
```

```
class Child(Parent1,Parent2):
```

```
    f6=models.CharField(max_length=64)
```

Note:

1. Multiple inheritance is also internally multi table inheritance only.
2. In multiple inheritance Parent classes should not contain any common field, otherwise we will get error.

Model Manager:

Model Manager can be used to interact with the database. By default Model Manager is available through the Model.objects property.i.e Model.objects is of type django.db.models.manager.Manager.

1) What is the purpose of Model Manager?

To interact with database

2. How to get Default Model Manager?

By using Model.objects property

3. Model Manager is what type?

`django.db.models.manager.Manager`

```
>>> from testapp.models import Employee
```

```
>>> type(Employee.objects)
```

```
<class 'django.db.models.manager.Manager'>
```

We can customize the default behaviour of Model Manager by defining our own Custom Manager.

How to define our own Custom Manager:

We have to write child class for `models.Manager`.

Whenever we are using `all()` method, internally it will call `get_queryset()` method.

To customize behaviour we have to override this method in our Custom Manager class.

Eg: To retrieve all employees data according to ascending order of `eno`, we have to define Custom Manager class as follows.

[models.py](#)

```
from django.db import models
class CustomManager(models.Manager):
    def get_queryset(self):
        # Create your models here.
        return super().get_queryset().order_by('eno')
class Employee(models.Model):

    eno=models.IntegerField()
```



```
ename=models.CharField(max_length=64)
```

```
esal=models.FloatField()
```

```
eaddr=models.CharField(max_length=256)
```

```
objects=CustomManager()
```

When ever we are using all() method it will always get employees in ascending order of eno

Based on our requirement we can define our own new methods also inside Custom Manager class.

```
class CustomManager(models.Manager):
```

```
    def get_queryset(self):
```

```
        return super().get_queryset().order_by('eno')
```

```
    def get_emp_sal_range(self,esal1,esal2):
```

```
        return super().get_queryset().filter(esal range=(esal1,esal2))
```

```
    def get_employees_sorted_by(self,param):
```

```
        return super().get_queryset().order_by(param)
```

Q) To Customize all() Method Behaviour, which Method we have to override inside Custom Manager Class?

get_queryset() Method

Q) In Custom Manager Class, is it Possible to define New Methods?

Yes

views.py

```
from django.shortcuts import render
```

```
def display_view(request):
```

```
    employees=Employee.objects.get_employees_sorted_by('esal')
```

```
    my_dict={'employees':employees}
```

```
    return render(request,'testapp/index.html',my_dict)
```

index.html

```
<!DOCTYPE html>
```

```
2) {% extends 'testapp/base.html'%}
```

```
3)    {%block body_block%}
```

```
4)    <h1>Employee Information Dash Board</h1><hr>
```

```
5)    <table border='2'>
```

```
6)    <thead>
```

```
7)    <th>Employee Number</th>
```

```
8)    <th>Employee Name</th>
```

```
9)    <th>Employee Salary</th>
```

```
10)   <th>Employee Address</th>
```

```
11)
```

```
12)   </thead>
```

```
13)   {%for emp in employees %}
```

```
14)   <tr>
```

```
15)   <td>{{emp.eno}}</td>
```

```

16) <td>{{emp.ename}}</td>
17) <td>{{emp.esal}}</td>
18) <td>{{emp.eaddr}}</td>
20) </tr>
21) {%endfor%}
22) </table><br><br><br>
23) {%endblock%}

```

5) Proxy Model Inheritance:

- For the same Model we can provide a customized view without touching the database is possible by using Proxy Model Inheritance.
- In this inheritance a separate new table won't be created and the new model also pointing to the same old table.

1) class Employee(models.Model):

2) fields

4) class ProxyEmployee(Employee):

5) class Meta:

6) proxy=True

- Both Employee and ProxyEmployee are pointing to the same table only.
- In the admin interface if we add a new record to either Employee or ProxyEmployee, then automatically those changes will be reflected to other model view.

Demo Application:

models.py

```
from django.db import models 2)

3)     class CustomManager1(models.Manager):
4)         def get_queryset(self):
5)             return super().get_queryset().filter(esal gte=15000)
6)
7)     class CustomManager2(models.Manager):
8)         def get_queryset(self):
9)             return super().get_queryset().order_by('ename')
10)
11)    class CustomManager3(models.Manager):
12)        def get_queryset(self):
13)            return super().get_queryset().filter(eno lt=1000)
14)
15)    # Create your models here.
16)    class Employee(models.Model):
17)        eno=models.IntegerField()
18)        ename=models.CharField(max_length=64)
19)        esal=models.FloatField()
20)        eaddr=models.CharField(max_length=256)
21)        objects=CustomManager1()
22)
23)    class ProxyEmployee(Employee):
```

```
24) objects=CustomManager2()

25) class Meta:

26) proxy=True

27)

28) class ProxyEmployee2(Employee):

29) objects=CustomManager3()

30) class Meta:

31) proxy=True
```

admin.py

```
1) from django.contrib import admin

2) from testapp.models import Employee,ProxyEmployee,ProxyEmployee2

4) # Register your models here.

5) class EmployeeAdmin(admin.ModelAdmin):

6)     list_display=['eno','ename','esal','eaddr']

7) class ProxyEmployeeAdmin(admin.ModelAdmin):

8)     list_display=['eno','ename','esal','eaddr']

9)

10) class ProxyEmployee2Admin(admin.ModelAdmin):

11)     list_display=['eno','ename','esal','eaddr']

12)

13) admin.site.register(Employee,EmployeeAdmin)

14) admin.site.register(ProxyEmployee,ProxyEmployeeAdmin)

15) admin.site.register(ProxyEmployee2,ProxyEmployee2Admin)
```

views.py

```
1) from django.shortcuts import render
2) from testapp.models import Employee,ProxyEmployee,ProxyEmployee2
3) # Create your views here.
4) def display_view(request):
5)     # employees=Employee.objects.all()
6)     # employees=ProxyEmployee.objects.all()
7)     employees=ProxyEmployee2.objects.all()
8)     my_dict={'employees':employees}
9)     return render(request,'testapp/index.html',my_dict)
```

index.html

```
1) <!DOCTYPE html>
2) {% extends 'testapp/base.html'%}
3)     {%block body_block%}
4)         <h1>Employee Information Dash Board</h1><hr>
5)         <table border='2'>
6)             <thead>
7)                 <th>Employee Number</th>
8)                 <th>Employee Name</th>
9)                 <th>Employee Salary</th>
10)                <th>Employee Address</th>
11)            </thead>
12)            {%for emp in employees %}
```

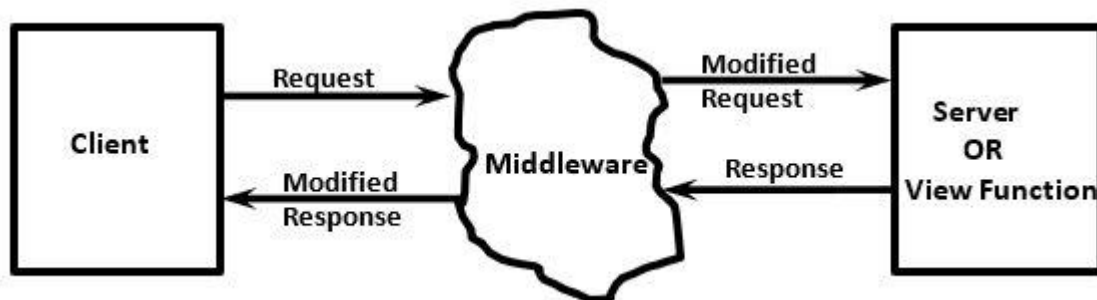
```
13) <tr>
14) <td>{{emp.eno}}</td>
15) <td>{{emp.ename}}</td>
16) <td>{{emp.esal}}</td>
17) <td>{{emp.eaddr}}</td>
18) </tr>
19) {%endfor%}
20) </table><br><br><br>
21) {%endblock%}
```

Working with Django Middleware

Middleware:

Middleware is a framework of hooks into Django's request/response processing. It is a light, low level 'plugin' system for globally altering Django's input or output.

If we want to perform any activity at the time of pre processing of the request or post processing of the request then we should go for middleware.



Client → Request → Middleware → modifiedrequest

Server → Response → Middleware → modifiedresponse

Django contains several inbuilt middlewares which are configured inside settings.py

```
1) MIDDLEWARE = [  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
]
```

All these middlewares will be executed before and after processing of every request.

- 1) SecurityMiddleware provides security enhancements like SSL Redirects (like from http request to https request) etc.
- 2) SessionMiddleware enables session support.
- 3) CommonMiddleware provides a common set of features like adding slash at the end of the URL.

- 4) **CsrfViewMiddleware** is responsible to verify whether POST request has **csrf_token** or not.
- 5) **AuthenticationMiddleware** is responsible to add user attribute to the request object.

If we comment this middleware in settings.py then we cannot access user attribute in our view function. If we are trying to access you will get error.

```
print(request.user)
```

AttributeError at /first/

'WSGIRequest' object has no attribute 'user'

Note: Middlewares are applicable for every incoming request and for every outgoing response.

Middleware Structure:

Based on our requirement we can define our own customized middlewares.

Every customized middleware is a python class and it is the child class of object, contains 2 mandatory methods and 3 optional methods.

class LoginMiddleware(object):

```
def _init_(self, get_response):
```

#one time configuration and initialization on start-up, get_response is a reference to previous middleware response

```
self.get_response = get_response
```

def __call__(self, request):

```
#This code will be executed before the view(and other middleware) is called  
response = self.get_response(request) #It triggers next phase
```

```
#This code will be executed after the view(and other middleware) is  
called return response # to finish middleware sequence
```

```
def process_view(self,request,view_func,view_args,view_kwargs):
```

```
# Logic will be executed before a call to View
```

```
# Gives access to the view itself and arguments
```

```
def process_exception(self,request,exception):
```

```
#Logic will be executed if an exception/error occurs in the view
```

```
def process_template_response(self,request,response):
```

```
#Logic is executed after view is called.
```

It is required to alter the response itself to perform additional logic on it like modifying context or template.

Demo Application for Custom Middleware Execution Flow:

middleware.py:(inside application folder)

```
1) class ExecutionFlowMiddleware(object):
2)     def __init__(self,get_response):
3)         self.get_response=get_response
4)
5)     def __call__(self,request):
6)         print('This line added at pre-processing of request')
7)         response=self.get_response(request)
8)         print('This line added at post-processing of request')
9)         return response
```

settings.py

```
1) MIDDLEWARE = []
2) ..... ,
3)     'testapp.middleware.ExecutionFlowMiddleware'
4) ]
```

views.py

```
1) from django.http import HttpResponse
2)
3) # Create your views here.
4) def welcome_view(request):
5)     print('This line added by view function')
6)     return HttpResponse('<h1>Custom Middleware Demo</h1>')
```

Results:

If we send a request in the server console we can see:

**This line added at pre-processing of
request This line added by view
function**

This line added at post-processing of request

Before and After processing every request middleware will be executed.

Execution Process for a Single Middleware Class:

- 1) _init_() method will be called only once at the time of server start-up.**
- 2) _call_() method will be called for every request.**
- 3) If we declare process_view() method then it will be called**
- 4) Inside _call_() method whenever we are using
self.get_response(request) then view function starts its execution**
- 5) If we declare process_exception() method, then it will be
executed if any exception/error occurs inside view function.**
- 6) View Method Finishes.**
- 7) If we declare process_template_response() then it will be executed
whenever view returns TemplateResponse.**

Middleware application to show information saying application under maintenance:

[middleware.py](#)

```
from django.http import HttpResponse
```

```
class AppMaintenanceMiddleware(object):
```

```
def init (self,get_response):
```

```
    self.get_response=get_response
```

```
def call \(self,request\):
```

```
    return HttpResponse('<h1>Currently Application under maintenance...  
plz try after 2 days!!!')
```

[setting.py](#)

```
MIDDLEWARE = [
```

```
    'testapp.middleware
```

```
]
```

[views.py](#)

```
from django.http import HttpResponse
```

```
# Create your views here.
```

```
def home\_page\_view\(request\):
```

```
    return HttpResponse('<h1>Hello This is from home page view</h1>')
```

Middleware application to show meaningful response if view function raises any error:

In this case we have to use `process_exception()` method which will be executed if view function raising any exception/error.

middleware.py

```
from django.http import HttpResponseRedirect

2) class ErrorMessageMiddleware(object):

3)     def init (self,get_response):

4)         self.get_response=get_response

6)         def call (self,request):

7)             return self.get_response(request)

9)         def process_exception(self,request,exception):

10)            return HttpResponseRedirect('<h1>Currently we are facing some technical problems
plz try after some time!!!</h1>')
```

settings.py

```
1) MIDDLEWARE = [

3)     'testapp.middleware.ErrorMessageMiddleware'
```

views.py

```
1) from django.http import HttpResponseRedirect

3) # Create your views here.

4) def home_page_view(request):

5)     print(10/0)

6)     return HttpResponseRedirect('<h1>Hello This is from home page view</h1>')
```

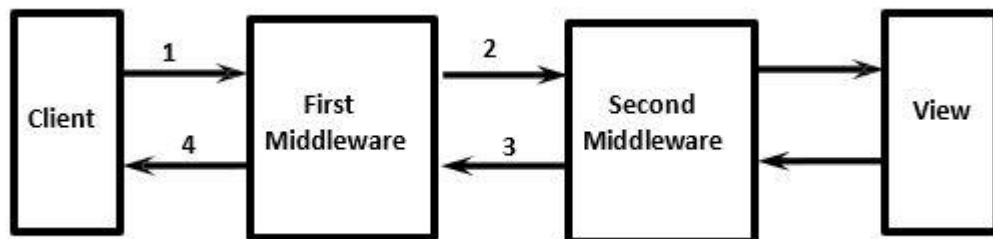
How to display raised exception information:

```
def process_exception(self,request,exception):
```

```
    return HttpResponse('<h1>Currently we are facing some technical  
problems plz try after some time!!!</h1><h2>Raised  
Exception:{}</h2><h2>Exception Message:{}</h2>'.format(exception. class .  
    name ,exception))
```

Configuration of multiple middleware classes:

We can configure any number of middlewares and all these middlewares will be executed according to order declared inside settings



[middleware.py](#)

```
class FirstMiddleware(object):
```

```
2)    def __init__ (self,get_response):
```

```
3)        self.get_response=get_response
```

```
5)    def __call__ (self,request):
```

```
6)        print('This line printed by FirstMiddleware at pre-processing of request')
```

```
        response=self.get_response(request)
```

```
        print('This line printed by FirstMiddleware at post-processing of  
request')
```

```
        return response
```

```
class SecondMiddleware(object):
```

```
    def __init__ (self,get_response):
```

```

        self.get_response=get_response
    def __call__(self,request):
        print('This line printed by SecondMiddleware at pre-processing of
request')
        response=self.get_response(request)
        print('This line printed by SecondMiddleware at post-processing of
request')
        return response

```

settings.py

```

MIDDLEWARE = [
    'testapp.middleware.FirstMiddleware',
]

```

[views.py](#)

```

def home_page_view(request):
    return HttpResponse('<h1>Hello This is from home page view</h1>')

```

In the Server Console:

This line printed by FirstMiddleware at pre-processing of request
 This line printed by SecondMiddleware at pre-processing of request
 This line printed by view function
 This line printed by SecondMiddleware at post-processing of request
 This line printed by FirstMiddleware at post-processing of request

Note: If we change the order of middlewares inside settings.py then the output at server console is:

This line printed by SecondMiddleware at pre-processing of request
 This line printed by FirstMiddleware at pre-processing of request
 This line printed by view function
 This line printed by FirstMiddleware at post-processing of request
 This line printed by SecondMiddleware at post-processing of request